BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC KINH TẾ TP HỒ CHÍ MINH UEH
TRƯỜNG CÔNG NGHỆ VÀ THIẾT KẾ

ĐỒ ÁN MÔN HỌC
XÂY DỰNG ỨNG DỤNG BÁN NÔNG SẢN GIẢI CỨU

RescueFarm

Học phần Phát triển ứng dụng Mobile

Chuyên ngành Công nghệ thông tin | Khóa K50

Giảng viên TS. Đặng Ngọc Hoàng Thành

Nhóm 7

Lại Thành Đạt
Phan Khắc Anh Tuấn
Nguyễn Phương Chinh
Nguyễn Tấn Khiêm
Trương Thành Đạt

Bản nháp chờ xác nhận MSSV lớp và phân công

Audit mã nguồn ngày 24 tháng 10 năm 2026

PHẠM VI VÀ CÁCH ĐỌC BẰNG CHỨNG

Báo cáo phân tích RescueFarm như một hệ thống thương mại điện tử giải cứu nông sản. Phần chính giải thích quan hệ giữa nghiệp vụ, giao diện Android, dữ liệu và backend; ảnh và kết quả kiểm thử được thu thập từ môi trường demo cô lập. Chương 1 giữ nội dung do nhóm cung cấp trong PDF; các chương sau được tổng hợp từ repository hiện tại.

HEAD nền 28adb48bb2e5e4ac8230379a1152f64a52251f5e. Working tree có thay đổi phục vụ thực nghiệm chưa commit và thay đổi prompt có sẵn được giữ nguyên. APK demo có package riêng và SHA-256 trong evidence. Không tự commit, push hoặc deploy; không sử dụng dữ liệu hay giao dịch production để tạo ảnh.

Quy ước đường dẫn: ui/, domain/, data/ và service/ được rút gọn từ app/src/main/java/com/rescuefarm/; layout và navigation thuộc app/src/main/res/; functions/ và cloudinary-worker/ tính từ root repository. Chương 1 giữ nguyên nội dung nhóm; nhận định về thiết bị vật lý trong bản nhóm chưa có bằng chứng ở lượt audit này, vốn chỉ dùng emulator.

Bảng 1 tổng hợp các mức bằng chứng sử dụng trong báo cáo.

Bảng 1 Các mức bằng chứng sử dụng trong báo cáo

Nhãn bằng chứng	Cách hiểu
Có code	Đã đọc đường triển khai; chưa đồng nghĩa runtime hoặc provider ngoài đã hoạt động.
Đã qua test	Bài kiểm thử đã chạy trong lượt audit, với phạm vi và log ghi rõ.
Đã thao tác trên emulator	Activity/Fragment thật đã chạy với dữ liệu demo; không suy rộng mọi nút.
Chưa xác minh tích hợp ngoài	OAuth, ngân hàng, upload, FCM hoặc deploy live chưa được thực nghiệm.
Thiếu hoặc chưa hoàn thiện	Thiếu thông tin nhóm, ca demo hay yêu cầu học thuật được nêu cụ thể.

MSSV, lớp cụ thể, nhiệm vụ được giao và tỷ lệ hoàn thành chờ nhóm xác nhận. Chưa có kết quả kiểm tra tương đồng UEH ở chế độ mặc định; bản báo cáo không cam kết dưới 20% khi chưa có minh chứng thật.

MỤC LỤC

PHẠM VI VÀ CÁCH ĐỌC BẰNG CHỨNG 2
MỤC LỤC 3
DANH MỤC HÌNH VÀ BẢNG 5
CHƯƠNG 1 TỔNG QUAN 6
1.1. Giới thiệu 6
1.2. Tính Cấp Thiết của Đề tài 6
1.3. Công Cụ Phát Triển Ứng Dụng Di động 6
1.4. Kiểm thử ứng dụng trên Android Emulator và Thiết bị 7
CHƯƠNG 2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 8
2.1 Bài toán và ranh giới hệ thống 8
2.2 Tác nhân và quyền sở hữu 9
2.3 Use case tổng thể 9
2.4 Phân rã chức năng và tiêu chí chấp nhận 11
2.5 Yêu cầu khám phá và chọn hàng 13
2.6 Yêu cầu giỏ hàng và checkout 13
2.7 Yêu cầu đơn hàng và giao nhận 14
2.8 Yêu cầu chiến dịch và tương tác cộng đồng 15
2.9 Yêu cầu giải cứu mở rộng 15
2.10 Yêu cầu Hũ giải cứu theo chính sách mới 16
2.11 Kiến trúc và thiết kế trách nhiệm 16
CHƯƠNG 3 THIẾT KẾ GIAO DIỆN 19
3.1 Nguồn ảnh và quy trình xác minh 19
3.2 Trang chủ và khám phá cho Guest 20
3.3 Chi tiết sản phẩm và giỏ Guest 21
3.4 Giỏ Customer và thông tin checkout 22
3.5 Quản lý sản phẩm và lô của Seller 23
3.6 Dashboard và chiến dịch Seller 24
3.7 Quản trị trên mobile 25
3.8 Giao diện Hũ và hạn giao Việt Nam 26
3.9 Hũ chờ duyệt và ghi chi 27
3.10 Responsive và giữ trạng thái 29
3.11 UI đọc lại danh mục khi offline 30
CHƯƠNG 4 XÂY DỰNG CƠ SỞ DỮ LIỆU 31
4.1 Mô hình dữ liệu tổng thể 31
4.2 Danh mục dữ liệu người dùng và hàng hóa 33
4.3 Product và Batch cùng bất biến tồn kho 33
4.4 Chiến dịch và phân bổ theo lô 34
4.5 Đơn hàng và snapshot giao dịch 35
4.6 Dữ liệu Hũ và sổ biến động 35
4.7 Dữ liệu giải cứu và các collection vận hành 36
4.8 SQLite và các bảng Room thực tế 37
4.9 TTL và đồng bộ cục bộ 38
4.10 Truy vấn tiêu biểu và index 39
4.11 Quan hệ hướng đối tượng và mapper 40
CHƯƠNG 5 XÂY DỰNG ỨNG DỤNG 42
5.1 Cấu trúc mã và môi trường build 42
5.2 Kết nối Firebase và phân quyền 42
5.3 Xác thực và Google Credential Manager 43
5.4 Triển khai Home và tìm kiếm 44
5.5 Triển khai chi tiết sản phẩm và giỏ 44
5.6 Triển khai checkout và xác nhận tiền 45
5.7 Triển khai Seller và vòng đời chiến dịch 45

5.8 Triển khai tương tác và quản trị 46
5.9 Triển khai Hũ từ tiền vào tới khoản chi 46
5.10 API vị trí và media 47
5.11 Đồng bộ và xử lý lỗi 47
5.12 Tích hợp Git và CI 48
CHƯƠNG 6 QUY TRÌNH THỰC THI VÀ KIỂM THỬ 49
6.1 Quy trình đặt hàng tổng hợp 49
6.2 Quy trình tài khoản và thêm sản phẩm 51
6.3 Quy trình Hũ và thí nghiệm thời gian 51
6.4 Phương pháp và môi trường kiểm thử 53
6.5 Kết quả Hũ tại các biên quan trọng 53
6.6 Kết quả responsive và offline 54
6.7 Rủi ro còn lại và kết luận kỹ thuật 55
PHỤ LỤC PHÂN CÔNG VÀ MINH CHỨNG GIT 56
Nguồn mã và lịch sử tích hợp 56
Bảng phân công chờ xác nhận của nhóm 57
TÀI LIỆU THAM KHẢO 58

DANH MỤC HÌNH VÀ BẢNG

Hình 1 Use case tổng thể của bốn vai trò 10
Hình 2 Phân rã chức năng theo nghiệp vụ hiện hành 12
Hình 3 Kiến trúc tổng thể với đường đọc offline và thao tác online 18
Hình 4 Trang chủ và khám phá trong phiên Guest với dữ liệu demo 20
Hình 5 Chi tiết Product với lô đang bán và giỏ Guest sau thao tác thêm hàng 21
Hình 6 Giỏ Customer và form checkout với hồ sơ tổng hợp 22
Hình 7 Danh sách sản phẩm và lô hàng thuộc Seller demo 23
Hình 8 Dashboard Seller và trạng thái danh sách chiến dịch của fixture 24
Hình 9 Dashboard và danh sách sản phẩm trên giao diện Admin mobile 25
Hình 10 Hũ đã giữ tiền và chuyến Seller đã nhận với deadline giờ Việt Nam 26
Hình 11 Chuyến đã gửi đúng hạn và trạng thái sau xác minh của Admin 27
Hình 12 Trạng thái PAID của chuyến chính và ledger giải phóng chuyến quá hạn riêng 28
Hình 13 Cùng form đăng nhập và cùng nội dung trước và sau xoay màn 29
Hình 14 Catalog online và dữ liệu đọc lại qua Room sau khi tiến trình khởi động lại offline 30
Hình 15 Quan hệ collection, snapshot giao dịch và cache SQLite 32
Hình 16 Quan hệ lớp rút gọn và trách nhiệm của các tầng 41
Hình 17 Luồng đặt hàng với kiểm tra tồn, QR hoặc COD và nhánh lỗi 50
Hình 18 Hoạt động Hũ với nhánh quá hạn và duyệt sau hạn 52

Bảng 1 Các mức bằng chứng sử dụng trong báo cáo 2
Bảng 2 Bài toán và ranh giới hệ thống 8
Bảng 3 Tác nhân và quyền sở hữu 9
Bảng 4 Phân rã chức năng và tiêu chí chấp nhận 13
Bảng 5 Yêu cầu giỏ hàng và checkout 14
Bảng 6 Yêu cầu Hũ giải cứu theo chính sách mới 16
Bảng 7 Nguồn ảnh và quy trình xác minh 19
Bảng 8 Danh mục dữ liệu người dùng và hàng hóa 33
Bảng 9 Product và Batch cùng bất biến tồn kho 34
Bảng 10 Chiến dịch và phân bổ theo lô 34
Bảng 11 Đơn hàng và snapshot giao dịch 35
Bảng 12 Dữ liệu Hũ và sổ biến động 36
Bảng 13 Dữ liệu giải cứu và các collection vận hành 36
Bảng 14 SQLite và các bảng Room thực tế 37
Bảng 15 TTL và đồng bộ cục bộ 38
Bảng 16 Truy vấn tiêu biểu và index 39
Bảng 17 Cấu trúc mã và môi trường build 42
Bảng 18 Xác thực và Google Credential Manager 43
Bảng 19 API vị trí và media 47
Bảng 20 Quy trình tài khoản và thêm sản phẩm 51
Bảng 21 Kết quả kiểm thử mới trên revision được audit 53
Bảng 22 Kết quả Hũ tại các biên quan trọng 54
Bảng 23 Kết quả responsive và offline 54
Bảng 24 Các merge gần HEAD để kiểm tra tích hợp 56
Bảng 25 Phân công cần nhóm xác nhận trước nộp 57

CHƯƠNG 1 TỔNG QUAN

1.1. Giới thiệu

Giải cứu nông sản không chỉ là đăng bán một mặt hàng cần tiêu thụ nhanh. Đây còn là bài toán kết nối đúng nguồn hàng với người mua, đồng thời cập nhật kịp thời số lượng và trạng thái giao dịch. Khi thông tin về sản phẩm và nguồn cung phân tán, người mua khó tìm được hàng phù hợp, còn người bán khó theo dõi quá trình tiêu thụ.

RescueFarm là ứng dụng Android thuộc lĩnh vực thương mại điện tử, được phát triển nhằm hỗ trợ quá trình kết nối này. Ứng dụng cho phép người bán giới thiệu sản phẩm, quản lý hàng hóa theo từng lô và tổ chức chiến dịch giải cứu; người mua có thể tìm kiếm sản phẩm, đặt hàng và theo dõi trạng thái giao dịch. Trong phạm vi đề tài, "giải cứu nông sản" hướng đến việc thúc đẩy tiêu thụ kịp thời những sản phẩm vẫn phù hợp để sử dụng.

Hệ thống phục vụ bốn nhóm người dùng: khách vãng lai, khách hàng, người bán và quản trị viên. Chức năng và quyền truy cập được tổ chức theo vai trò, hỗ trợ các nghiệp vụ từ tra cứu sản phẩm, quản lý đơn hàng đến kiểm duyệt nội dung. Qua đó, RescueFarm vận dụng kiến thức lập trình ứng dụng di động vào việc thiết kế và triển khai một hệ thống có người dùng, dữ liệu và quy trình nghiệp vụ cụ thể.

1.2. Tính Cấp Thiết của Đề tài

Nông sản chịu ảnh hưởng của tính mùa vụ và có thời gian bảo quản giới hạn. Khi tốc độ tiêu thụ không theo kịp nguồn cung, người bán có thể gặp tình trạng tồn hàng, suy giảm giá trị sản phẩm và phát sinh lãng phí. Do đó, việc hỗ trợ giới thiệu hàng hóa và kết nối người bán với người mua có ý nghĩa thiết thực đối với quá trình phân phối nông sản.

Một khó khăn nhức nhối khác là thông tin về địa điểm, số lượng còn lại và tiến độ tiêu thụ có thể được chia sẻ qua nhiều kênh. Điều này làm cho việc tra cứu, cập nhật tình trạng hàng hóa và theo dõi giao dịch trở nên thiếu tập trung. RescueFarm hướng đến giải quyết vấn đề đó bằng cách tổ chức thông tin sản phẩm, quản lý hàng hóa theo từng lô và theo dõi trạng thái giao dịch trên cùng một ứng dụng.

Về phương diện học tập, đề tài tạo cơ hội vận dụng kiến thức lập trình hướng đối tượng, thiết kế giao diện, tổ chức dữ liệu và kiểm thử phần mềm vào một bài toán thực tiễn. Quá trình phát triển RescueFarm cũng giúp làm rõ cách phối hợp giữa ứng dụng di động, dịch vụ lưu trữ dữ liệu và backend trong việc triển khai các chức năng của một hệ thống thương mại điện tử.

1.3. Công Cụ Phát Triển Ứng Dụng Di động

a) Giới thiệu sơ lược về Java

Java là ngôn ngữ lập trình hướng đối tượng, hỗ trợ tổ chức chương trình thành các lớp và đối tượng với những nguyên lý như đóng gói, kế thừa, trừu tượng và đa hình. Trong

RescueFarm, Java được sử dụng để xây dựng mô hình dữ liệu, xử lý nghiệp vụ và tương tác với giao diện XML. Mã nguồn được tổ chức theo MVVM kết hợp Repository nhằm tách phần hiển thị, quản lý trạng thái và truy cập dữ liệu, thuận tiện cho bảo trì và mở rộng.

b) Android SDK

Android SDK là bộ công cụ phục vụ phát triển ứng dụng Android, gồm các API nền tảng và công cụ biên dịch, đóng gói, gỡ lỗi, chạy thử. Các thành phần tiêu biểu gồm SDK Platform, Build Tools, Platform Tools và Android Emulator. Trong đề tài, Android SDK hỗ trợ triển khai giao diện, điều hướng, kết nối mạng và xử lý quyền truy cập như vị trí, thông báo.

c) Android Studio

Android Studio là môi trường phát triển tích hợp chính thức dành cho Android, cung cấp công cụ soạn thảo mã, thiết kế giao diện, quản lý thư viện và build bằng Gradle. Dự án sử dụng Android Studio để phát triển Java/XML, theo dõi lỗi qua Logcat và đóng gói APK. Các thư viện AndroidX, Material Components và Room hỗ trợ xây dựng giao diện, quản lý trạng thái và lưu dữ liệu cục bộ.

d) Firebase

Firebase cung cấp các dịch vụ hỗ trợ xác thực, lưu trữ dữ liệu và gửi thông báo. RescueFarm sử dụng Firebase Authentication cho đăng ký, đăng nhập; Cloud Firestore để quản lý dữ liệu như hồ sơ, sản phẩm, chiến dịch và đơn hàng; Firebase Cloud Messaging để hỗ trợ thông báo. Quyền truy cập được kiểm soát theo vai trò và các quy tắc bảo mật.

Bên cạnh Firebase, hệ thống sử dụng Cloudinary cho hình ảnh và backend Node.js/Express trên Render để xử lý những nghiệp vụ cần kiểm tra tập trung như đặt hàng, tồn kho và xác minh thanh toán.

1.4. Kiểm thử ứng dụng trên Android Emulator và Thiết bị

Kiểm thử nhằm đánh giá chức năng, giao diện và khả năng phản hồi của ứng dụng. Android Emulator cho phép chạy thử trên nhiều cấu hình màn hình và phiên bản hệ điều hành, hỗ trợ kiểm tra điều hướng, biểu mẫu và cách hiển thị khi thay đổi kích thước hoặc cỡ chữ.

Thiết bị Android thực tế được sử dụng để đánh giá thao tác, kết nối mạng và các chức năng phụ thuộc thiết bị như GPS, chọn ảnh, cấp quyền và nhận thông báo. Phạm vi kiểm thử cần bao phủ bốn nhóm người dùng, đặc biệt là quy trình đặt hàng, thanh toán, hủy đơn và xác nhận nhận hàng.

Ngoài kiểm tra giao diện, cần kết hợp kiểm thử đơn vị cho logic nghiệp vụ và Firestore Emulator để kiểm tra quyền truy cập dữ liệu. Kịch bản và kết quả thực nghiệm sẽ được trình bày trong chương kiểm thử của báo cáo.

CHƯƠNG 2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1 Bài toán và ranh giới hệ thống

RescueFarm tổ chức giao dịch nông sản còn phù hợp để sử dụng nhưng cần tiêu thụ sớm. Hàng hóa được giới thiệu ở cấp Product, còn thời hạn sử dụng và biến động số lượng thuộc ProductBatch. Bài toán trọng tâm là kết nối nhu cầu với một nguồn hàng cụ thể, kiểm tra lại khả năng bán khi giao dịch và lưu lịch sử sau khi hàng không còn khả dụng. Vì vậy, việc nhìn thấy sản phẩm trên màn hình không tạo ra quyền mua số lượng đang hiển thị.

Phạm vi chính là thương mại điện tử trên Android. Chiến dịch cung cấp nội dung giải cứu và điểm nhận; bình luận, reaction và report tạo tương tác gắn với nghiệp vụ. Những thành phần này không đủ để kết luận hệ thống là một mạng xã hội hoàn chỉnh. Flash Sale, Hộp bí ẩn và Hũ được triển khai như các cơ chế giải cứu bổ sung, mỗi cơ chế có trạng thái, điều kiện tồn kho và thanh toán riêng.

Ranh giới ứng dụng bao gồm Android Java/XML, Firebase Authentication, Cloud Firestore, backend Express trong functions/ và Worker ký media trong cloudinary-worker/. Tên thư mục functions không có nghĩa các nghiệp vụ đang chạy trên Firebase Cloud Functions. Các màn quản trị là Fragment của cùng ứng dụng mobile; không có bằng chứng để mô tả một web admin thay thế.

Nguồn triển khai: domain/model/Product.java, ProductBatch.java; ui/rescue/; ui/fund/; functions/commerce.js, rescue-programs.js và rescue-fund.js. README và final.md được dùng để nhận diện ý định ban đầu, sau đó đối chiếu các đường dẫn này. Riêng firestore.schema.json vẫn có mục promotions và customerPromotionWallets cũ, nên không được dùng độc lập như danh sách chức năng hiện hành.

Bảng 2 tổng hợp bài toán và ranh giới hệ thống.

Bảng 2 Bài toán và ranh giới hệ thống

Nội dung	Trong phạm vi hiện tại	Giới hạn cần giải thích
Mua bán	Khám phá, chọn lô, giỏ, checkout, đơn, giao nhận	Giá và tồn được kiểm tra lại trên backend
Nội dung	Campaign và tương tác theo ngữ cảnh	Không suy rộng thành nền tảng tin tức hoặc mạng xã hội
Giải cứu	Flash Sale, Hộp bí ẩn, Hũ thiện nguyện	Không gộp các cơ chế thành cùng một kiểu thanh toán
Ngoại tuyến	Cache đọc và giỏ Guest	Không thực hiện thanh toán hay giữ kho offline

2.2 Tác nhân và quyền sở hữu

Guest là phiên chưa đăng nhập, không phải một document users có quyền mặc định. Giỏ Guest được lưu cục bộ; những API cho Guest yêu cầu thông tin phiên hoặc token biên nhận thích hợp. Customer có hồ sơ Auth và document users; giỏ đăng nhập, địa chỉ và danh sách đơn phải truy vấn theo chủ thể đang hoạt động. Seller quản lý dữ liệu của mình khi tài khoản ACTIVE và trạng thái Seller phù hợp. Admin thực hiện tác vụ quản trị trên mobile nhưng vẫn chịu kiểm tra quyền ở từng đường truy cập.

MainActivity theo dõi thay đổi role/status/sellerStatus từ document người dùng và khôi phục vai trò. Điều hướng hoặc việc ẩn nút chỉ hỗ trợ trải nghiệm; chúng không phải biện pháp bảo mật đủ dùng. Client Rules kiểm tra truy cập từ Android SDK. Với API sử dụng Admin SDK, hàm auth trong server.js kiểm tra Firebase ID token và hồ sơ ACTIVE; endpoint tiếp tục kiểm tra role và quyền sở hữu vì Admin SDK không dựa vào client Rules để quyết định quyền.

Ở SnapshotLiveData, ownerId ràng buộc với UID đang đăng nhập. Khi tài khoản thay đổi, listener được tháo và giá trị rỗng được phát trước khi gắn nguồn tiếp theo. Đây là lớp bảo vệ hiển thị cần thiết cho cache riêng, bổ sung cho truy vấn DAO theo ownerId hoặc sellerId. Không có cơ sở suy ra toàn bộ dữ liệu cục bộ đã được mã hóa chỉ từ cơ chế scope này.

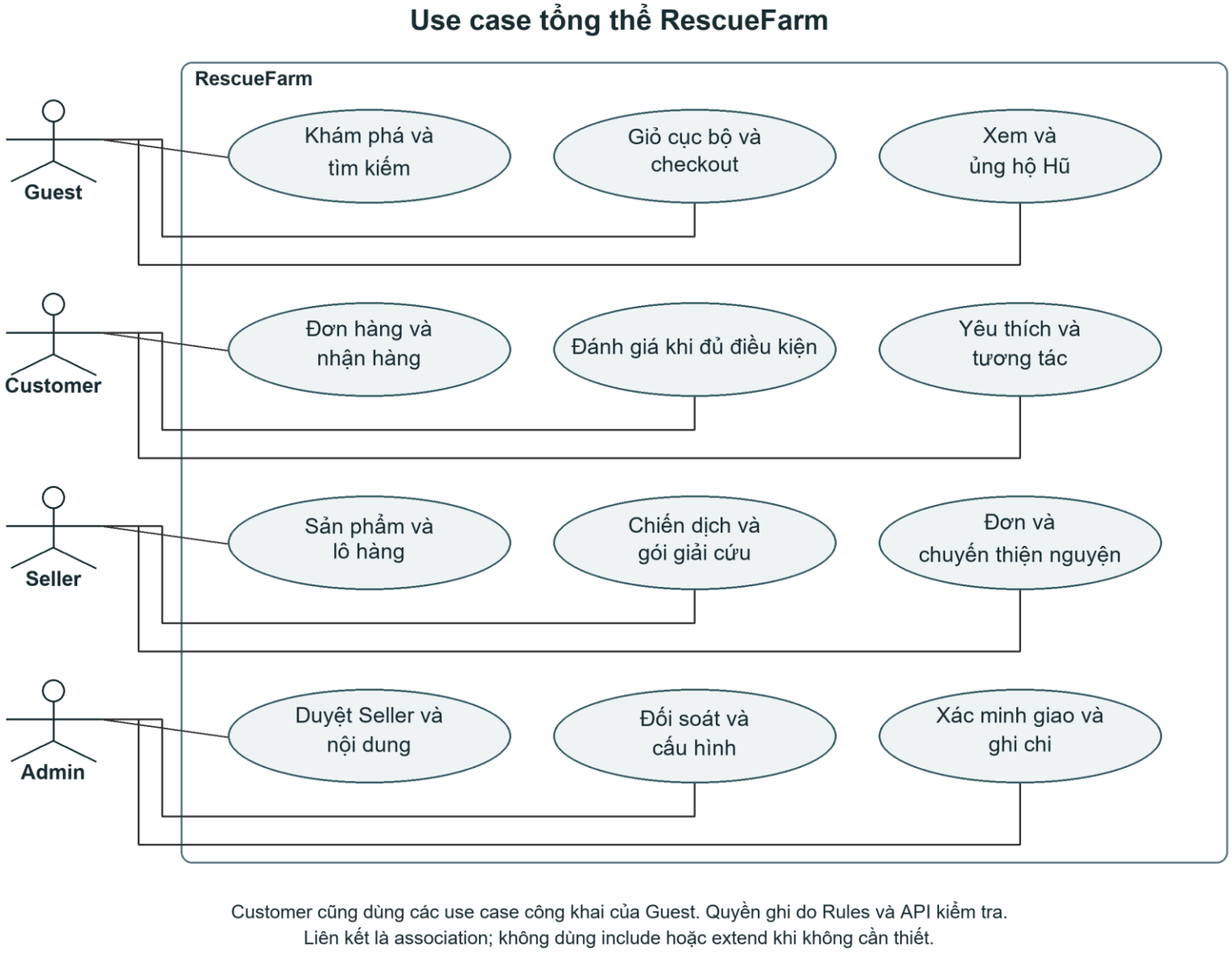
Bảng 3 tổng hợp tác nhân và quyền sở hữu.

Bảng 3 Tác nhân và quyền sở hữu

Actor	Dữ liệu hoặc hành động đại diện	Điều kiện trọng yếu
Guest	Catalog công khai, giỏ local, đơn Guest, biên nhận Hũ	Token phiên và quyền truy cập API tương ứng
Customer	customerCarts, addresses, orders, review	UID trùng chủ thể; review phải đủ điều kiện
Seller	products, productBatches, campaigns, deliveries của mình	ACTIVE, APPROVED và sellerId đúng
Admin	Seller applications, moderation, fund verify/pay	Role ADMIN; kiểm tra trạng thái nghiệp vụ trước ghi

2.3 Use case tổng thể

Hình 1 gom bốn tác nhân trong một ranh giới RescueFarm. Các use case được bố trí theo vùng nghiệp vụ để dễ đọc; đường nối biểu diễn quyền tham gia, không biểu diễn thứ tự chạy. Customer tiếp tục sử dụng nhóm chức năng công khai. Không vẽ include hoặc extend chỉ để làm sơ đồ phức tạp hơn khi không có quan hệ bắt buộc hoặc mở rộng được xác định rõ.



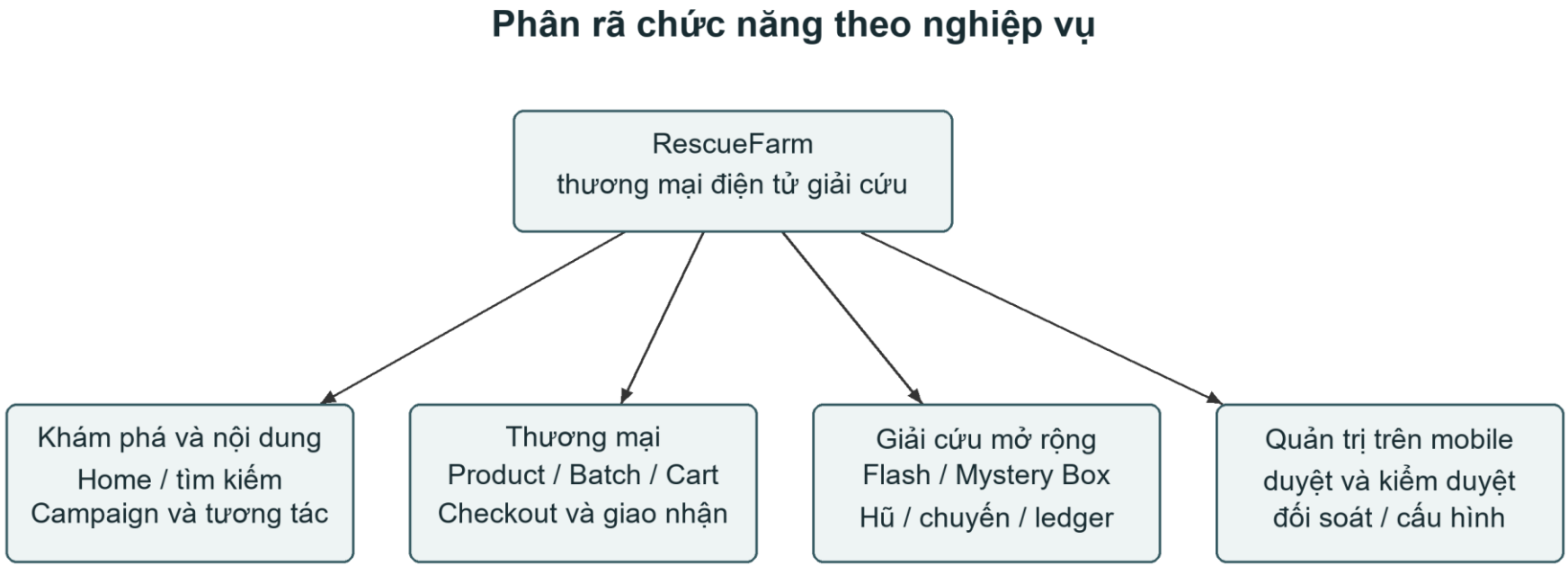
Hình 1 Use case tổng thể của bốn vai trò

Use case “checkout” không đồng nghĩa thanh toán đã được ngân hàng xác nhận. Tương tự, “ghi chi” trong Hũ không phải API chuyển tiền ngân hàng. Hai cách đặt tên này giúp phân biệt thao tác người dùng khởi tạo với sự kiện bên ngoài mà hệ thống chỉ có thể nhận biết qua webhook hoặc chứng từ.

Đối chiếu: main_navigation.xml, ui/navigation/ActorUiPolicy.java, FirebaseUserRepository, functions/server.js và các router nghiệp vụ. Bằng chứng thao tác trên emulator chỉ bao phủ tập màn đã chụp; không tự biến các association trong sơ đồ thành kết luận mọi nút đã được thao tác đầu cuối.

2.4 Phân rã chức năng và tiêu chí chấp nhận

Hình 2 chia hệ thống theo bốn nhóm liên quan với nhau: khám phá nội dung, giao dịch, giải cứu mở rộng và quản trị. Phân rã này phù hợp với các package UI và module backend nhưng không yêu cầu mỗi hộp tương ứng duy nhất một class. Repository và service được dùng chung để tránh sao chép chính sách tồn kho giữa nhiều màn hình.



Chức năng cũ promotions, wallet ưu đãi và bài đăng tổng quát không được khôi phục chỉ vì tên collection hoặc tài liệu lịch sử còn tồn tại.

Hình 2 Phân rã chức năng theo nghiệp vụ hiện hành

Bảng 4 tổng hợp phân rã chức năng và tiêu chí chấp nhận.

Bảng 4 Phân rã chức năng và tiêu chí chấp nhận

Nhóm	Tiêu chí chấp nhận có thể quan sát	Bằng chứng cần đối chiếu
Khám phá	Tìm kiếm trả kết quả phù hợp dữ liệu lô đang bán	DiscoveryEngine, test JVM và ảnh Guest
Giao dịch	Retry không giữ kho thêm; sai tồn bị từ chối	checkoutReceipts, transaction, suite API
Giải cứu	Dùng tồn từ lô thật; không lấy hàng đã hết hạn	rescue-programs, rescue-fund và test emulator
Quản trị	Thao tác đúng vai trò, có kiểm tra dữ liệu và lịch sử	Rules, API và ảnh Admin mobile

2.5 Yêu cầu khám phá và chọn hàng

Người mua cần nhận diện sản phẩm, giá theo đơn vị, lô còn hạn và điểm nhận trước khi thêm hàng. DiscoveryEngine nhận sản phẩm, chiến dịch, danh sách batch theo Product và tọa độ Seller. Hàm search kết hợp điều kiện văn bản, giá, tỉnh, tồn tối thiểu, thời hạn, khoảng cách và sắp xếp. Khi bộ lọc cần khoảng cách mà không có tọa độ hợp lệ, kết quả tương ứng không được giả định có khoảng cách bằng không.

Việc chuẩn hóa văn bản và kết hợp dữ liệu lô cho phép tìm kiếm nhất quán hơn so với tìm trên tên sản phẩm đơn lẻ. Các phép sắp xếp về giá hoặc lượng bán có chú ý đơn vị; so sánh kg với hộp như cùng một đại lượng sẽ gây hiểu sai. Đây là logic lọc và tính khoảng cách trong code, không có bằng chứng về một thuật toán AI gợi ý sản phẩm.

Đầu vào của luồng là bộ lọc và dữ liệu public đã tải; đầu ra là DiscoveryResult có Product hoặc Campaign, batch ứng viên và thông tin khoảng cách khi có. Ngoại lệ gồm quyền vị trí bị từ chối, tọa độ không hợp lệ, dữ liệu chưa tải hoặc lô không còn bán được. UI cần biểu diễn tình trạng này thay vì biến lỗi mạng thành kết luận không có hàng.

Nguồn: ui/home/DiscoveryQuery.java, DiscoveryQueryState.java, DiscoveryEngine.java, DiscoveryFragment.java và service/location/. Test JVM xác nhận những trường hợp đã mã hóa; ảnh màn khám phá chỉ xác nhận màn chạy được với fixture, chưa chứng minh độ chính xác GPS ngoài thực địa.

2.6 Yêu cầu giỏ hàng và checkout

Một dòng giỏ phải liên kết Product với Batch và số lượng. Guest sử dụng GuestCartRepository cùng Room; Customer sử dụng FirebaseCartRepository với document customerCarts theo UID. Đăng nhập có thể kích hoạt chuyển giỏ qua CartMigrationService/Worker và receipt chống lặp phía backend. Vì giỏ có thể được

đọc lâu trước checkout, giá trên giỏ chỉ có ý nghĩa tham khảo cho tới khi backend đối chiếu lại.

Checkout nhận danh sách item, hình thức nhận, phương thức thanh toán, thông tin người nhận và requestId. Backend commerce.js chuẩn hóa lựa chọn, kiểm tra Seller/Product/Batch/Campaign, tính giá và phí, sau đó giữ tồn và ghi snapshot trong transaction. requestId cùng fingerprint ngăn một retry với cùng nội dung tạo thêm đơn; nếu dùng lại mã với nội dung khác, request phải bị coi là xung đột.

Luồng lỗi cần xử lý rõ gồm giỏ rỗng, quantity không hợp lệ, tồn thay đổi, lô hết hạn, Seller không khả dụng, thông tin nhận thiếu hoặc phương thức không phù hợp. Client không được sửa số dư hay tồn để vượt qua lỗi. Người dùng phải sửa lựa chọn hoặc tải lại quote. Cơ chế càng mua càng giảm đã bị loại bỏ không được đưa trở lại khi diễn giải trường quantityDiscount còn phục vụ dữ liệu lịch sử.

Bảng 5 tổng hợp yêu cầu giỏ hàng và checkout.

Bảng 5 Yêu cầu giỏ hàng và checkout

Bước	Kết quả	Điều kiện kiểm tra lại
Chọn dòng giỏ	productId, batchId, quantity, selected	Batch bán được tại thời điểm chọn
Quote	Tổng tiền và phí dự kiến	Giá hiện hành, phương thức, điểm nhận
Checkout transaction	Đơn, item snapshot, giữ kho, receipt	Tồn và trạng thái còn hợp lệ khi commit
Nhận kết quả	Chuyển màn đơn hoặc QR	Retry dùng lại danh tính request đúng nội dung

2.7 Yêu cầu đơn hàng và giao nhận

Order mô tả giao dịch; paymentStatus mô tả tiền, còn status mô tả tiến trình nghiệp vụ. Không thể dùng việc đơn đã tạo để suy ra ngân hàng đã nhận tiền. payments và shipments tách thông tin xác minh hoặc giao nhận theo orderId, trong khi orders/items lưu tên, đơn giá và số lượng tại lúc đặt. Snapshot bảo vệ khả năng giải thích lịch sử sau khi Seller đổi tên hoặc giá sản phẩm.

Với BANK_TRANSFER, QR chứa số tiền và mã tham chiếu. SePay xử lý callback qua xác thực và transaction chống lặp trước khi cập nhật trạng thái. COD hoặc tiền tại điểm đi theo chính sách phương thức tương ứng; chúng không được diễn giải như callback QR. Hủy hoặc hết hạn phải giải phóng đúng phần tồn đang giữ, không xóa lịch sử đơn và không biến lô đã hết hạn thành lô bán được.

Ở mobile, Customer theo dõi đơn của mình; Seller theo dõi đơn thuộc cửa hàng và các bước giao nhận. Đối soát đơn là nghiệp vụ riêng để làm rõ các tình huống tiền và hàng

không đồng nhất. Báo cáo không coi thao tác "đã hoàn tiền" hoặc lưu chứng từ là hệ thống tự thực hiện lệnh chuyển khoản ra ngoài.

Nguồn: ui/order/, FirebaseOrderRepository, domain/model/Order.java, functions/commerce.js, sepay.js, order-reconciliation.js và batch-expiry.js. Suite Firestore/API kiểm tra trạng thái, quyền và retry theo các ca cụ thể. Bộ ảnh hiện có chưa minh chứng một chuyến giao thực tế với nhân viên vận chuyển.

2.8 Yêu cầu chiến dịch và tương tác cộng đồng

Campaign gắn mục tiêu với từng Batch qua batchTargets. Một Product có thể có nhiều lô với hạn khác nhau, nên gắn Campaign chỉ với Product sẽ làm mất thông tin nguồn hàng và hạn sử dụng. reservedByBatch và rescuedByBatch theo dõi phần đã giữ và đã hoàn thành; tổng số lượng của chiến dịch phải được giải thích cùng các bộ đếm theo lô, không chỉ bằng một con số tổng.

Chiến dịch có thời gian hoạt động và điểm cứu trợ cố định hoặc di động. Vị trí hiển thị phục vụ tìm kiếm và hướng dẫn nhận hàng; mở ứng dụng bản đồ bằng geo intent không chứng minh có Google Maps SDK hoặc theo dõi tài xế trực tiếp. Khi thời gian kết thúc hoặc hàng hết hạn, backend và UI phải đánh giá lại khả dụng, đồng thời giữ dữ liệu lịch sử cần đối soát.

Bình luận và reaction là subcollection gắn Campaign; review gắn giao dịch mua đủ điều kiện. Report cung cấp kênh báo cáo nội dung cho Admin. Việc có các collection này không cho phép suy ra có chat thời gian thực, livestream hoặc bài đăng xã hội tổng quát còn được hỗ trợ. Tài liệu và màn cũ phải được đối chiếu với điều hướng hiện hành trước khi đưa vào demo.

Nguồn: FirebaseCampaignRepository, FirebaseCampaignEngagementRepository, FirebaseEngagementRepository, functions/campaign-inventory.js, campaign-lifecycle.js và Rules vùng campaigns/comments/reactions/reviews/reports.

2.9 Yêu cầu giải cứu mở rộng

Flash Sale và Hộp bí ẩn vẫn có Fragment, ViewModel và backend rescue-programs.js. Cả hai sử dụng hàng từ Batch thật; một offer không được tạo thêm tồn vật lý. rescueClaims mô tả cam kết/giữ suất và quá trình nhận hàng. Chính sách hiện tại nhấn mạnh nhận tại điểm và thanh toán tại điểm, khác với việc thanh toán QR cho một đơn hàng thương mại thông thường.

Hộp bí ẩn che một phần nội dung theo trải nghiệm sản phẩm nhưng không được dùng sự bất định để che thông tin thiết yếu về nhận hàng và giá. Flash Sale giới hạn thời gian; việc hết thời gian hoặc hết suất phải được kiểm tra ở backend khi nhận claim. Các trạng thái đang giữ, đã nhận, đã hoàn tất và hủy có tác động tới tồn theo từng loại nghiệp vụ.

rescueImpact tổng hợp kết quả sau khi xác nhận nhận hàng và tách đơn vị vật lý. Không cộng kg, lít và hộp thành cùng một chỉ số khối lượng. Báo cáo không chuyển số liệu

fixture thành lượng nông sản đã cứu ngoài đời, cũng không quy đổi thành phát thải tránh được khi chưa có phương pháp đo.

Bằng chứng có code và suite emulator nằm ở ui/rescue/, functions/rescue-programs.js, rescue-policy.js, rescue-impact.js và các test tương ứng. Bộ fixture chụp ảnh chưa tạo offer/claim hoàn chỉnh; vì vậy hình của các màn này, nếu rỗng, chỉ chứng minh trạng thái không có dữ liệu chứ không chứng minh luồng claim đầu cuối.

2.10 Yêu cầu Hũ giải cứu theo chính sách mới

Hũ nhận ủng hộ qua mã và QR riêng; webhook hợp lệ tự ghi donation PAID. Khi dùng chung tài khoản đã kết nối của Seller, mã thuộc Hũ và mã thuộc đơn hàng vẫn đi theo các mapping khác nhau. Khóa giao dịch ngân hàng dùng chung ngăn một callback đồng thời cộng tiền Hũ và trả tiền đơn. Không có nút xác nhận tiền ủng hộ thủ công trong luồng chuẩn.

Ở thời điểm phân bổ t, điều kiện hạn Batch là expiryDate lớn hơn t cộng 24 giờ cộng một phút, đồng thời không vượt t cộng 48 giờ. Batch gắn Campaign cần Campaign ACTIVE, đúng Seller và đơn vị, startDate phù hợp và endDate vượt mốc giao cùng khoảng đệm. idleHours 12 hoặc 24 là thời gian không có giao dịch mua thành công; đây là điều kiện khác với hạn dùng và hạn giao.

Mỗi chuyến mới giữ cả tiền và hàng, có allocationPolicyVersion bằng 2, deliveryWindowHours bằng 24 và deadline bằng allocatedAt cộng 24 giờ. Nhận chuyến hoặc nhận FCM muộn không đặt lại deadline. RESERVED và ACCEPTED quá hạn chưa gửi ảnh bị hủy; DELIVERED đúng hạn được chờ Admin xét. VERIFY chốt kho nhưng tiền còn reserved; PAY chỉ ghi khoản đã chuyển ngoài app sau khi có số tiền, mã và ảnh chứng từ phù hợp.

Bảng 6 tổng hợp yêu cầu hũ giải cứu theo chính sách mới.

Bảng 6 Yêu cầu Hũ giải cứu theo chính sách mới

Đại lượng	Ý nghĩa	Không được đánh đồng
available	Tiền có thể phân bổ trong sổ Hũ	Số dư ngân hàng dùng chung
reserved	Tiền đã giữ cho các chuyến chưa ghi chi	Tiền Seller đã thực nhận
spent	Khoản chi đã được ghi sau xác minh	API tự chuyển tiền ra ngân hàng
RELEASE	Trả phần giữ về available	Hoàn tiền ngân hàng cho người ủng hộ

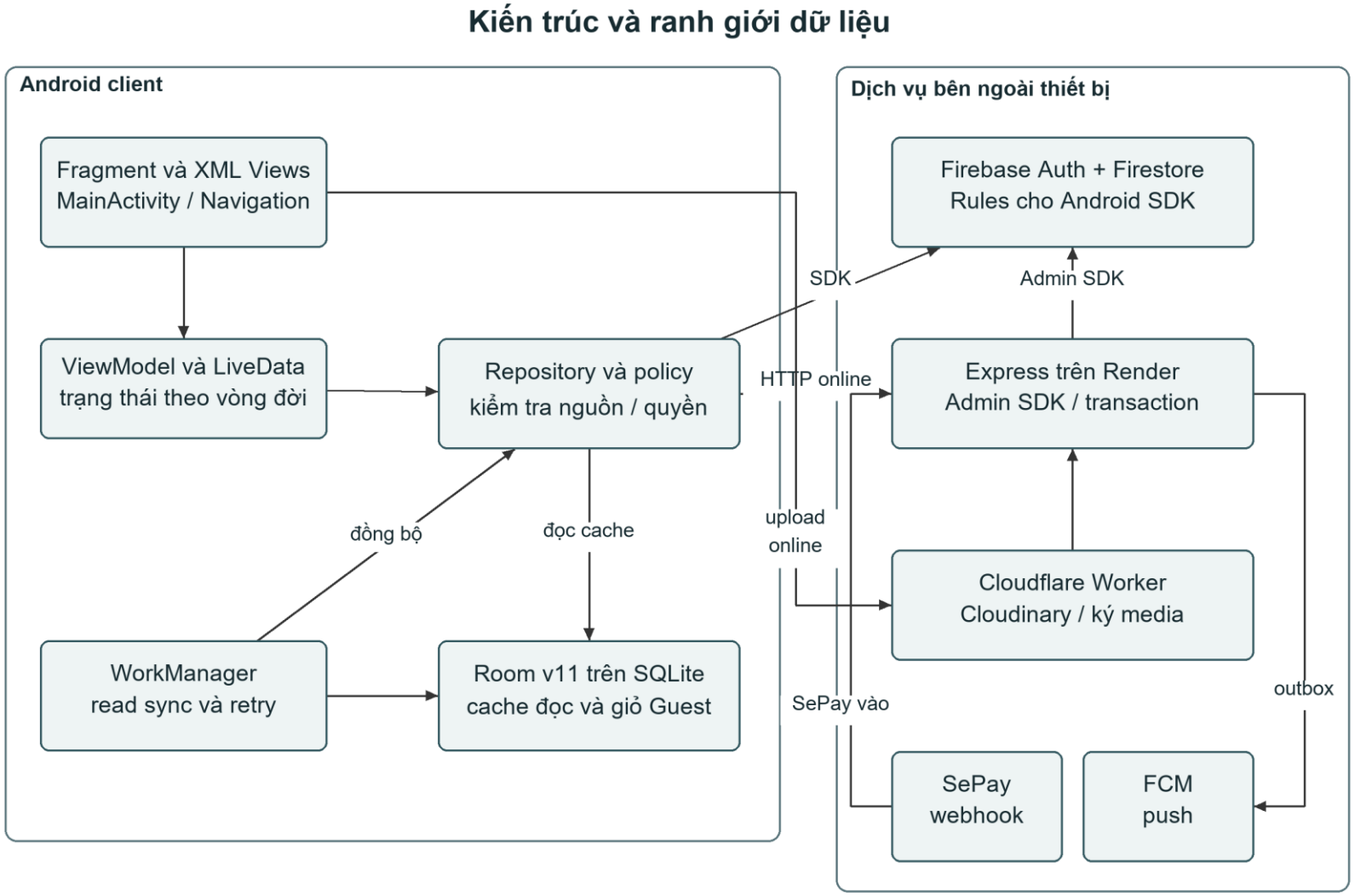
2.11 Kiến trúc và thiết kế trách nhiệm

Hình 3 thể hiện ranh giới client/server cùng hai đường dữ liệu: đọc có cache cục bộ và ghi nghiệp vụ cần online. Fragment xử lý View, điều hướng và sự kiện; ViewModel

cung cấp trạng thái để UI quan sát; Repository phối hợp nguồn Firebase, Room hoặc backend. Domain và service giữ những quy tắc có thể dùng lại, như tính khả dụng của lô và chuẩn hóa số lượng.

WorkManager chủ động làm mới phần cache đủ điều kiện khi có mạng. Đồng bộ đọc không phải cơ chế phát lại mọi thao tác nghiệp vụ. Backend dùng transaction cho các bước tác động đồng thời tới tiền, kho hoặc snapshot; Cloudinary Worker giữ trách nhiệm ký và kiểm tra quyền media. FCM là kênh nhắc người dùng, còn document và API mới là nguồn trạng thái khi mở màn hình.

Thiết kế phân lớp giúp định vị lỗi nhưng không làm biến mất độ phức tạp: một số Fragment và server.js vẫn lớn, nhiều trạng thái được triển khai trong cùng module. Hướng cải tiến là tách theo use case với hợp đồng rõ, duy trì regression test và tránh chuyển bất biến tài chính sang client chỉ để đơn giản hóa màn hình.



Hình 3 Kiến trúc tổng thể với đường đọc offline và thao tác online

CHƯƠNG 3 THIẾT KẾ GIAO DIỆN

3.1 Nguồn ảnh và quy trình xác minh

Các hình giao diện trong chương này là PNG chụp từ Activity Android đang chạy trên emulator-5580 API 33. APK được build từ HEAD 28adb48 cùng phần bổ sung phục vụ báo cáo chưa commit. Package demo có hậu tố .reportdemo, không cài đè package thường. Auth và Firestore dùng project demo-rescuefarm, HTTP trỏ 10.0.2.2:3008; dữ liệu catalog mang tiền tố report- và tên DEMO. Việc kết nối emulator được cấu hình riêng cho từng SDK và backend [3].

ReportScreenshotEvidenceTest mở destination thật, chờ UI, kiểm tra NavController rồi dùng UiAutomation.takeScreenshot. Đường dẫn ảnh gốc, vai trò, ý nghĩa, APK hash và giới hạn được lưu trong screenshots-manifest.json. Các ảnh không được dựng từ HTML, Canvas layout hoặc AI. Việc instrumentation mở trực tiếp destination được ghi rõ; nó xác nhận màn hình và dữ liệu, chưa chứng minh tất cả đường bấm từ drawer hoặc bottom navigation.

Mỗi cụm ảnh minh họa một luận điểm. Ảnh màn rỗng chỉ được dùng để giải thích trạng thái không có dữ liệu; không gọi nó là hoàn thành nghiệp vụ đầu cuối. Hình Hũ lấy trạng thái qua module thực và harness local, trong đó kiểm tra tồn tại ảnh Cloudinary được stub. Không có ảnh bàn giao thật, upload thật hoặc chuyển tiền thật trong bộ chứng cứ.

Bảng 7 tổng hợp nguồn ảnh và quy trình xác minh.

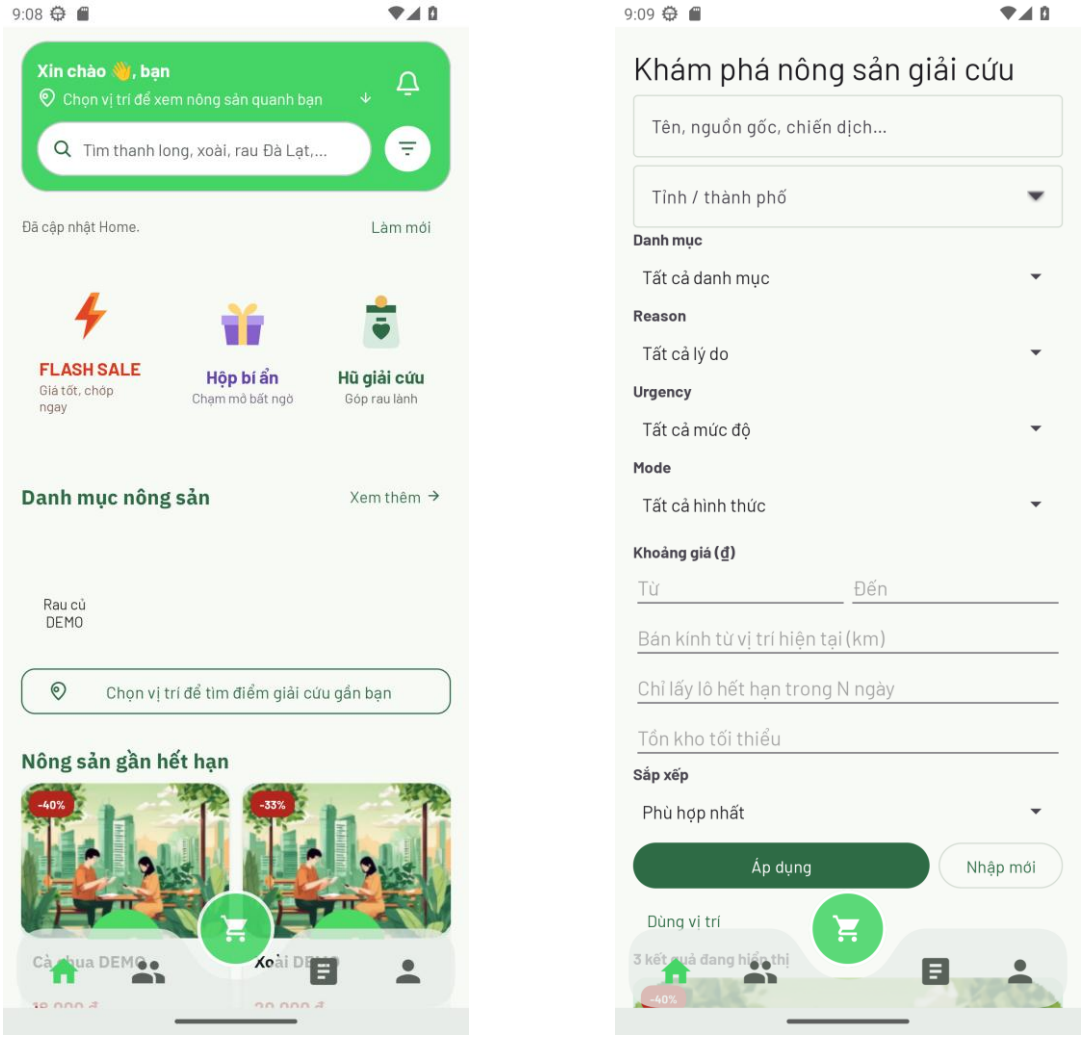
Bảng 7 Nguồn ảnh và quy trình xác minh

Thành phần môi trường	Cấu hình đã dùng	Giá trị chứng minh
Android	emulator riêng, package reportdemo	Không dùng phiên đăng nhập có sẵn
Firebase	Auth 9098, Firestore 8088, demo-rescuefarm	Dữ liệu và danh tính giả lập
HTTP	Server repo qua launcher local 3008	Logic repo; không chứng minh Render live
Media và tiền	Không upload/chuyển tiền thật; HEAD ảnh stub ở Hũ	Kiểm tra logic transition, giới hạn provider rõ ràng

3.2 Trang chủ và khám phá cho Guest

GuestHomeFragment sử dụng HomeViewModel cùng dữ liệu Product, Category và Campaign để tạo các nhóm nội dung. Trang chủ là điểm khám phá sản phẩm và chiến dịch; những phần thiếu fixture có thể hiển thị trạng thái trống thay vì số liệu giả. DiscoveryFragment chuyển tiêu chí lọc thành DiscoveryQuery để DiscoveryEngine đánh giá trên các dữ liệu đã tải.

Hình 4 ghi lại giao diện đang chạy trong môi trường demo.



(a) guest-home (b) guest-discovery

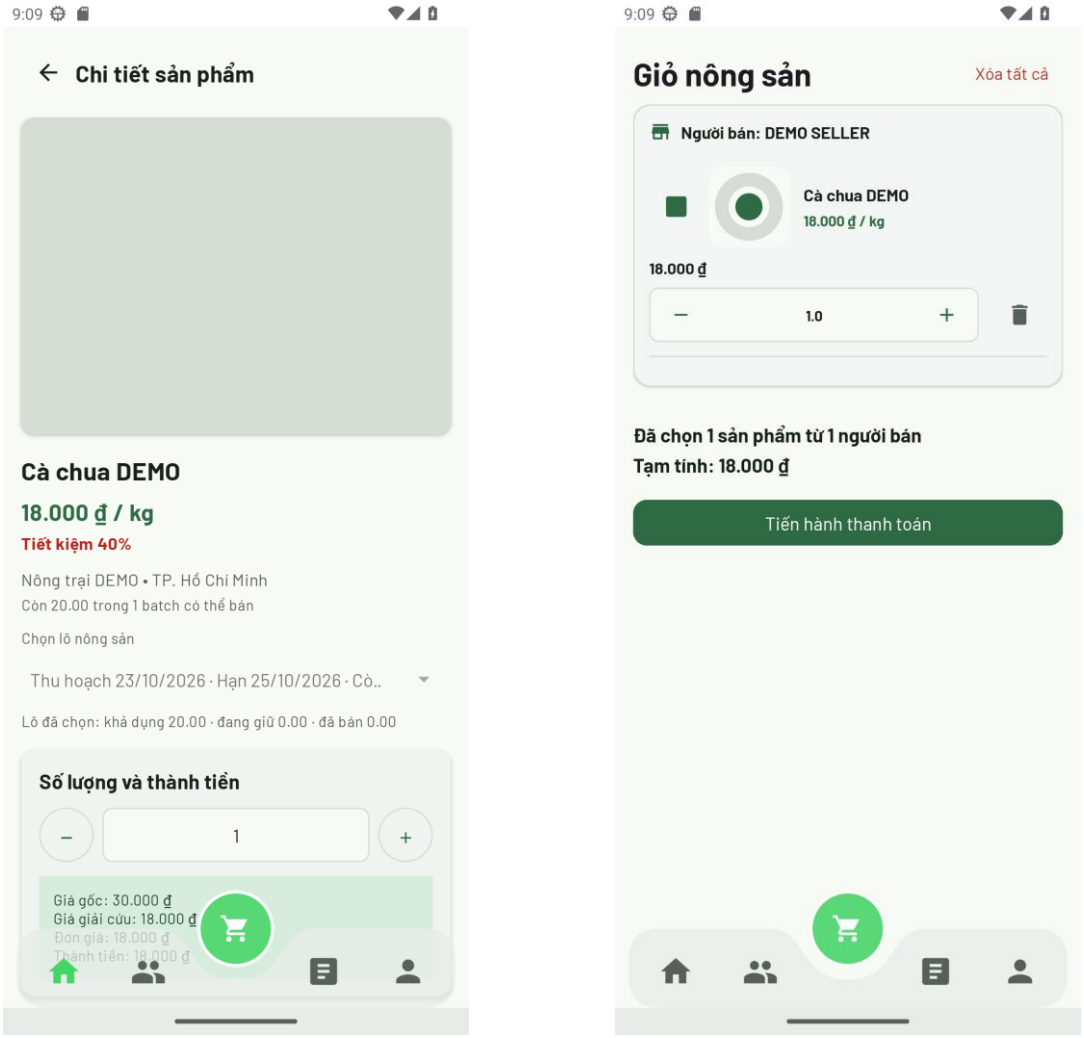
Hình 4 Trang chủ và khám phá trong phiên Guest với dữ liệu demo

Ở cụm ảnh trên, danh tính chưa đăng nhập và dữ liệu DEMO được giữ nhất quán. Kết quả tìm kiếm cần được hiểu trong phạm vi catalog đã tải và điều kiện lô đang bán. Quyền GPS, vị trí thật và độ chính xác geocoder chưa được chứng nhận bằng ảnh này. Các ca JVM của DiscoveryEngine là nguồn bổ sung cho bộ lọc và thứ tự; không suy từ một ảnh danh sách sang kết luận mọi tổ hợp bộ lọc đều qua test.

3.3 Chi tiết sản phẩm và giỏ Guest

ProductDetailFragment nhận productId, batchId và tùy chọn ngữ cảnh Campaign. Màn chi tiết hiển thị giá theo đơn vị và tồn bán được; bộ chọn số lượng gồm nút giảm, ô nhập và nút tăng. Java tham chiếu các ID pricingQuantityInput, quantityDecrease, quantityIncrease và addToCartButton, nên chúng phải tồn tại trong cả layout mặc định lẫn biến thể rộng.

Hình 5 ghi lại giao diện đang chạy trong môi trường demo.



(a) guest-product (b) guest-cart

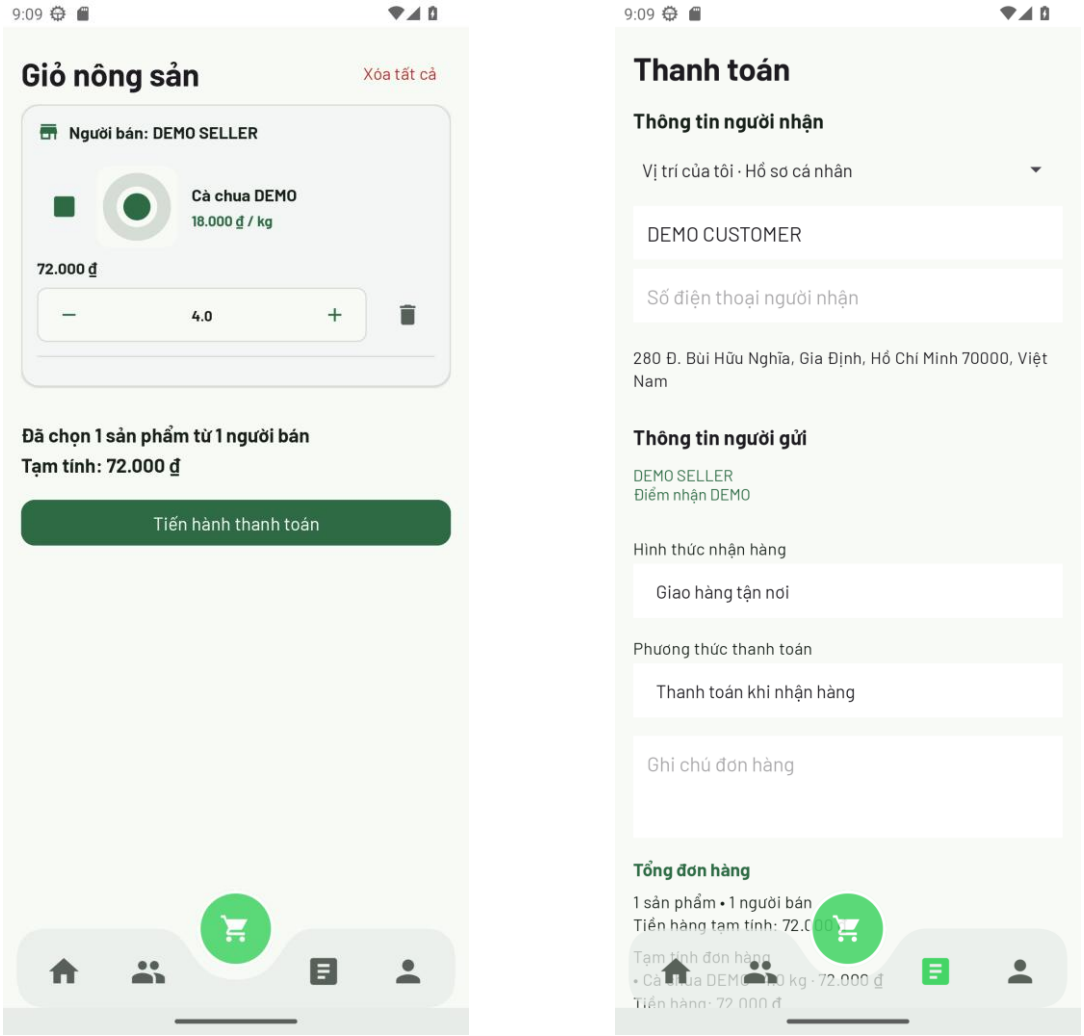
Hình 5 Chi tiết Product với lô đang bán và giỏ Guest sau thao tác thêm hàng

Ảnh giỏ được lấy sau khi instrumentation thực hiện click trên addToCartButton của màn Product thật. Đây là minh chứng thao tác trong ứng dụng, không phải gán tên sản phẩm vào một layout độc lập. Giá, số lượng và dòng giỏ vẫn cần revalidate khi chuyển bước; không dùng tồn trong ảnh làm cam kết giữ hàng. Những nhãn giảm giá theo chênh lệch originalPrice/rescuePrice không đồng nghĩa còn chương trình càng mua càng giảm.

3.4 Giỏ Customer và thông tin checkout

Sau đăng nhập Auth Emulator, MainActivity và AuthViewModel khôi phục vai trò Customer. Giỏ sử dụng dữ liệu của report-customer và cơ chế chuyển giỏ Guest hiện hành; vì vậy số lượng có thể phản ánh cả fixture ban đầu và hàng đã thêm trước đó. CartFragment nhóm dòng theo Seller, giữ thông tin lựa chọn và đưa tổng tạm tính tới CheckoutViewModel.

Hình 6 ghi lại giao diện đang chạy trong môi trường demo.



(a) customer-cart (b) customer-checkout

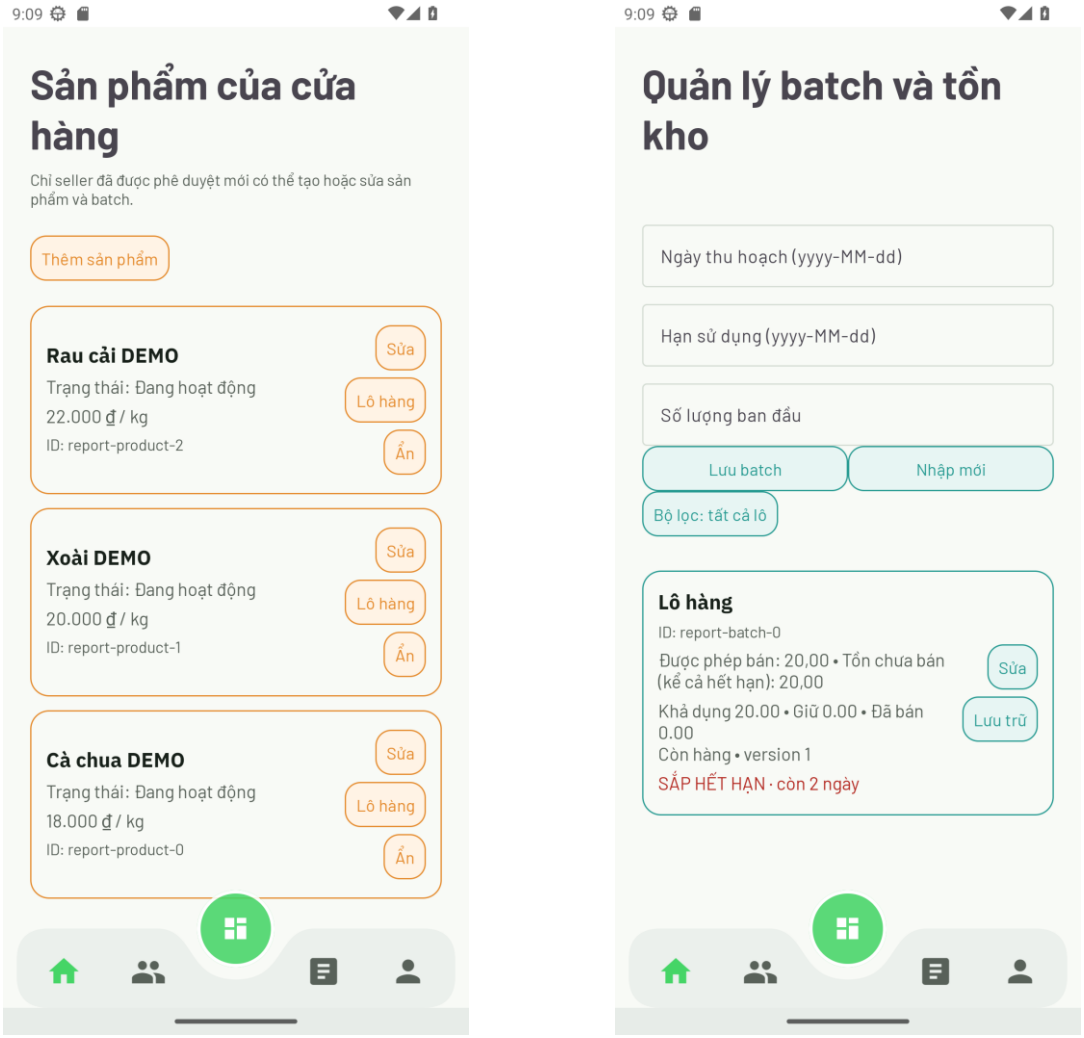
Hình 6 Giỏ Customer và form checkout với hồ sơ tổng hợp

Form checkout phối hợp dữ liệu hồ sơ, địa chỉ, hình thức nhận và phương thức thanh toán. Các con số trên ảnh thuộc môi trường demo, chưa phát sinh tiền thật. Màn còn có phần tổng đơn và hành động ở dưới vùng cuộn; ảnh phần trên không chứng minh nút cuối đã được thao tác. Thử nghiệm toàn quy trình đặt đơn/QR ở provider ngoài vẫn được phân biệt với suite API và hình form.

3.5 Quản lý sản phẩm và lô của Seller

SellerProductListFragment và BatchManagerFragment giúp Seller quản lý hàng thuộc cửa hàng của mình. Product giữ nội dung chung; danh sách lô cho biết hạn, tồn và trạng thái cụ thể. Nhập lô mới khác với sửa lịch sử một lô đã có biến động. Domain ProductBatch từ chối các thay đổi không hợp lệ và giữ invariant giữa initial, available, reserved và sold.

Hình 7 ghi lại giao diện đang chạy trong môi trường demo.



(a) seller-products (b) seller-batches

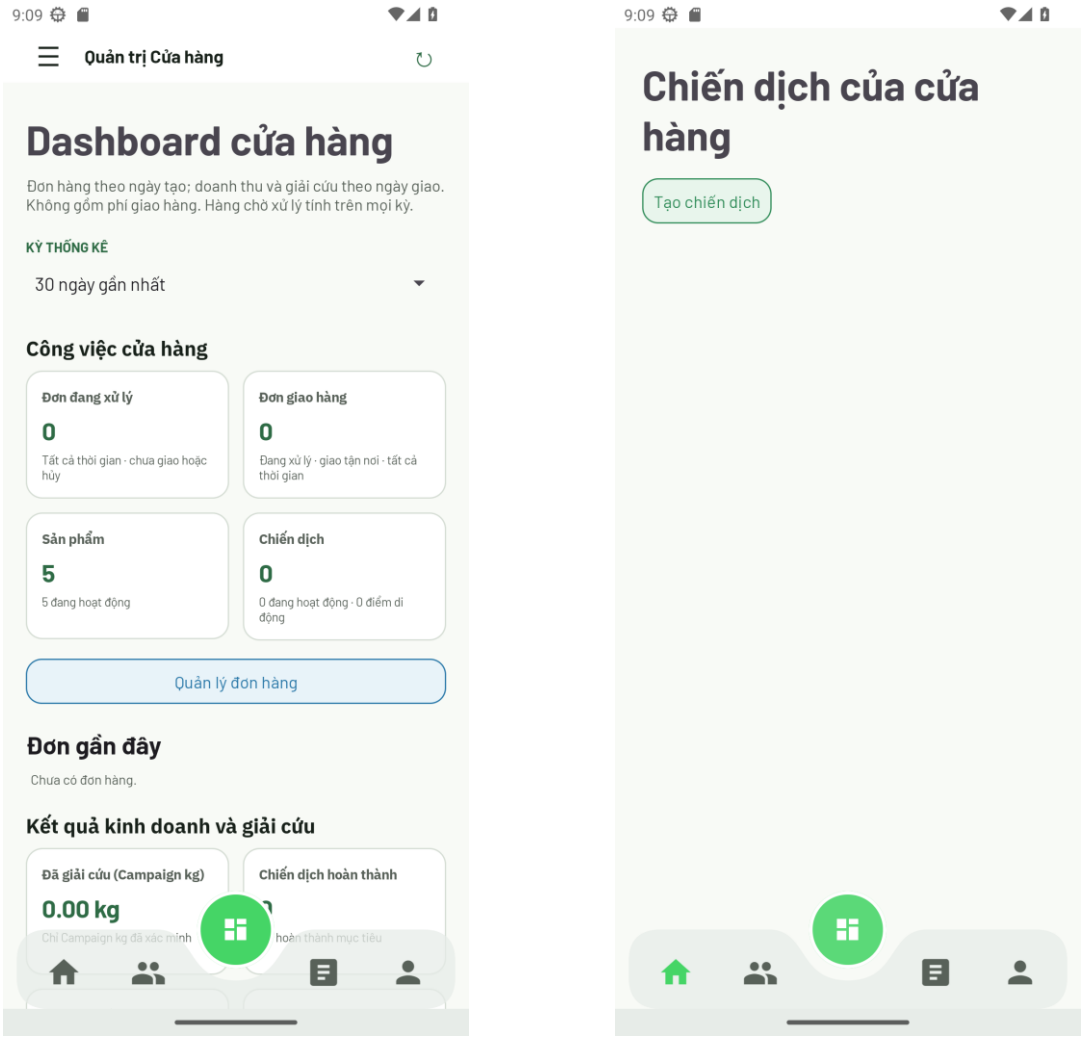
Hình 7 Danh sách sản phẩm và lô hàng thuộc Seller demo

Hai màn trong cụm ảnh lấy dữ liệu từ repository thật trên emulator. Bộ ảnh xác nhận khả năng đọc danh sách và hiển thị dữ liệu thuộc Seller; chưa ghi nhận thao tác upload ảnh hoặc tạo mới sản phẩm qua toàn bộ form. Quyền sở hữu không chỉ dựa trên việc màn đang hiển thị tên Seller: Rules và backend còn kiểm tra sellerId, trạng thái tài khoản và phiên bản tồn khi cập nhật.

3.6 Dashboard và chiến dịch Seller

SellerDashboardFragment tổng hợp thông tin phục vụ vận hành và cung cấp đường tới các module quản lý. SellerCampaignListFragment trình bày những Campaign thuộc cửa hàng. Dữ liệu chiến dịch trong seed catalog ban đầu chưa được tạo, nên trạng thái trống là kết quả đúng với fixture và phải được đọc theo ngữ cảnh đó.

Hình 8 ghi lại giao diện đang chạy trong môi trường demo.



(a) seller-dashboard (b) seller-campaigns

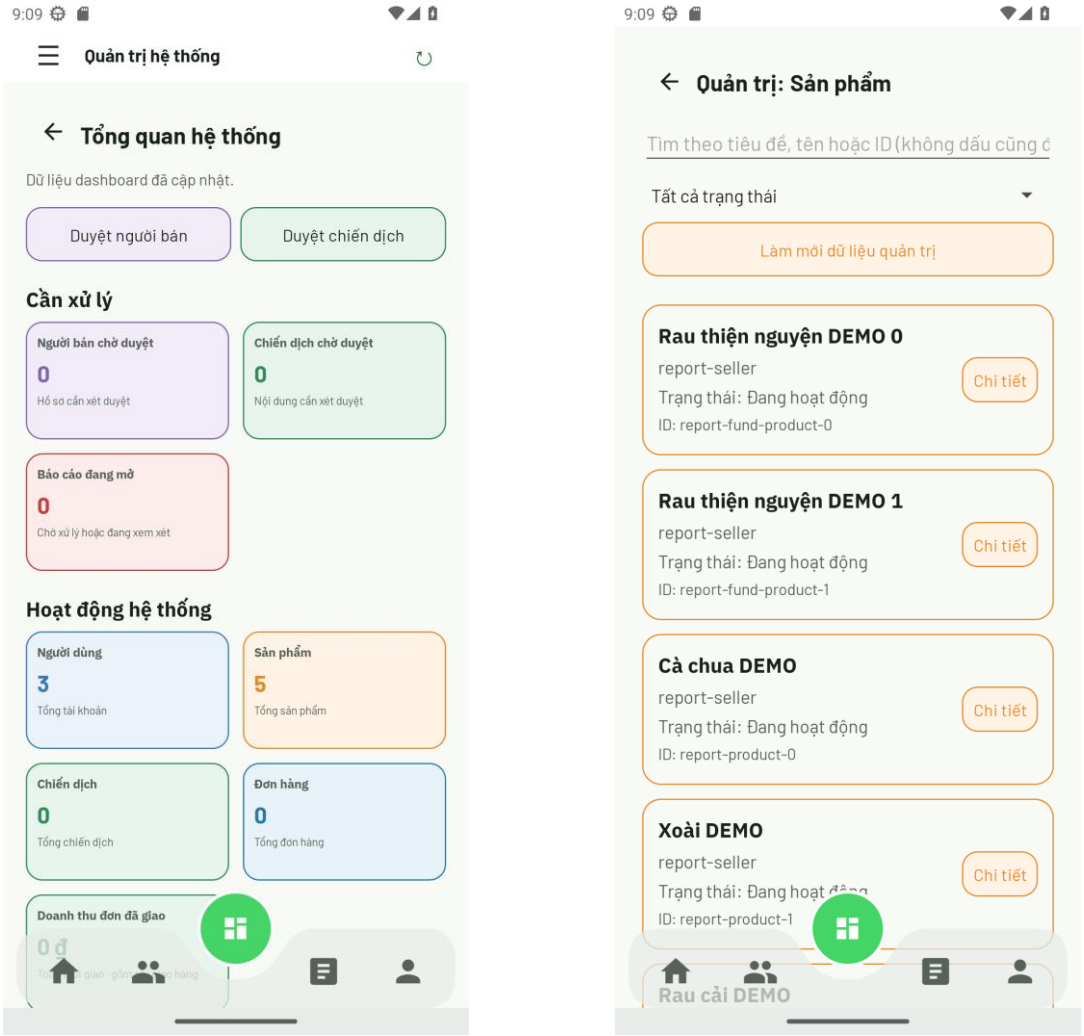
Hình 8 Dashboard Seller và trạng thái danh sách chiến dịch của fixture

Không có cơ sở dùng ảnh danh sách trống để kết luận Campaign end-to-end đã hoàn thành. Bằng chứng code gồm CampaignViewModel, FirebaseCampaignRepository và campaign-inventory/campaign-lifecycle; bằng chứng transaction đến từ suite Firestore/API. Hạn chế này giúp phân biệt màn đã thao tác với module mới chỉ có kiểm thử logic trong lượt audit.

3.7 Quản trị trên mobile

AdminDashboardFragment và AdminResourceFragment nằm trong ứng dụng Android, dùng AdminViewModel và FirebaseAdminRepository. Dashboard cung cấp tổng quan; resource screen phân trang và lọc theo Section. Tác vụ phê duyệt Seller, moderation Product/Campaign, xử lý report, cấu hình phí và đối soát cần được trình bày đúng quyền và dữ liệu mà API cho phép.

Hình 9 ghi lại giao diện đang chạy trong môi trường demo.



(a) admin-dashboard (b) admin-products

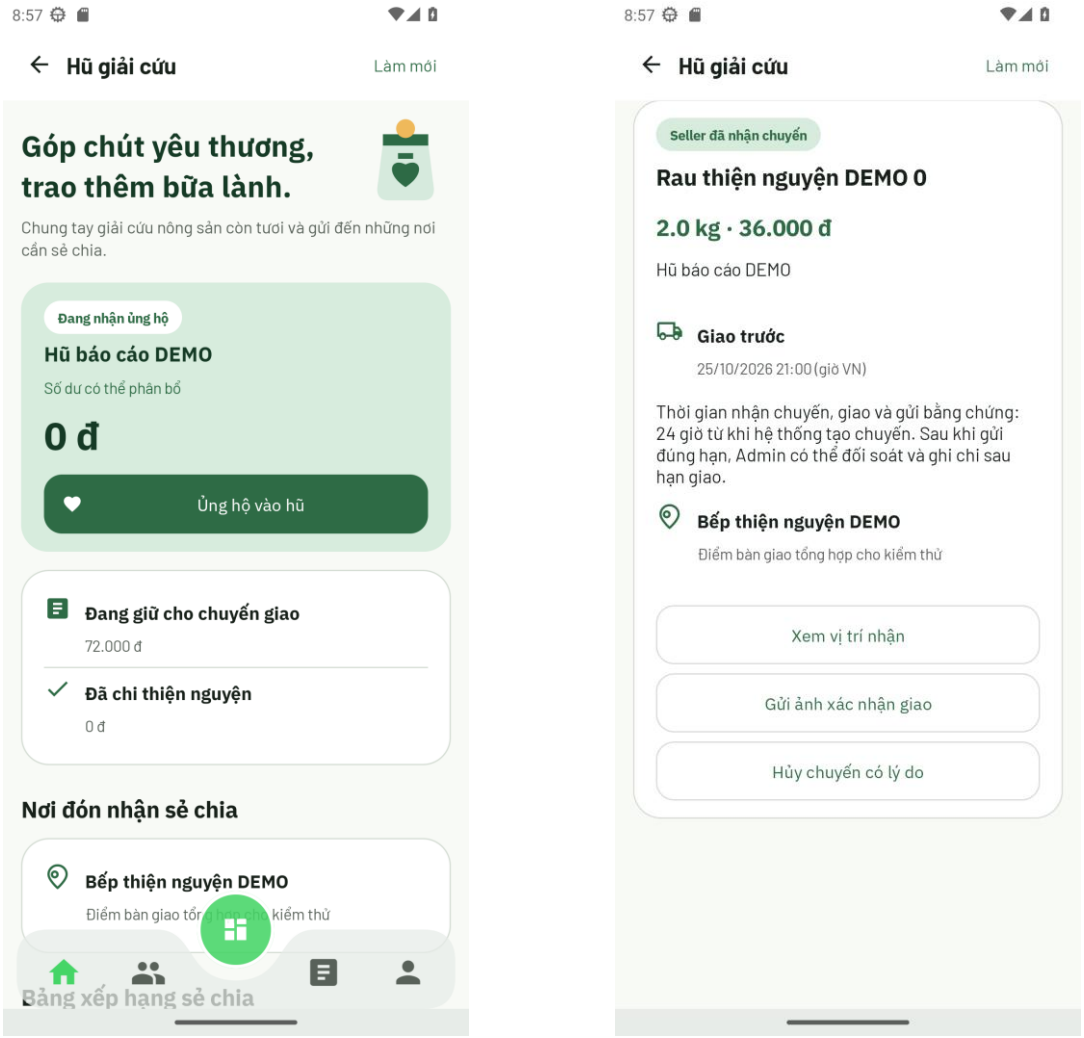
Hình 9 Dashboard và danh sách sản phẩm trên giao diện Admin mobile

Ảnh xác nhận vai trò Admin Emulator và dữ liệu quản trị đã tải được trong app. Lượt chụp mở destination trực tiếp và đọc tài nguyên; không thao tác phê duyệt tài khoản hoặc ẩn nội dung chỉ để tạo ảnh. Hệ thống không có self-registration Admin; quyền demo được seed bằng Admin SDK trong project giả lập, khác với việc sửa badge trên ảnh.

3.8 Giao diện Hũ và hạn giao Việt Nam

RescueFundFragment tải dữ liệu công khai hoặc Admin tùy vai trò rồi tải chuyến cho Seller/Admin. Màn hiển thị số tiền có thể phân bổ, đang giữ, đã chi, nơi nhận và bảng xếp hạng. RescueFundTime định dạng timestamp bằng Asia/Ho_Chi_Minh và thêm "giờ VN". Vì vậy múi giờ máy ảo không quyết định ngày giờ hiển thị trên màn Hũ.

Hình 10 ghi lại giao diện đang chạy trong môi trường demo.



(a) fund-reserved-seller-top (b) fund-accepted-seller-jobs

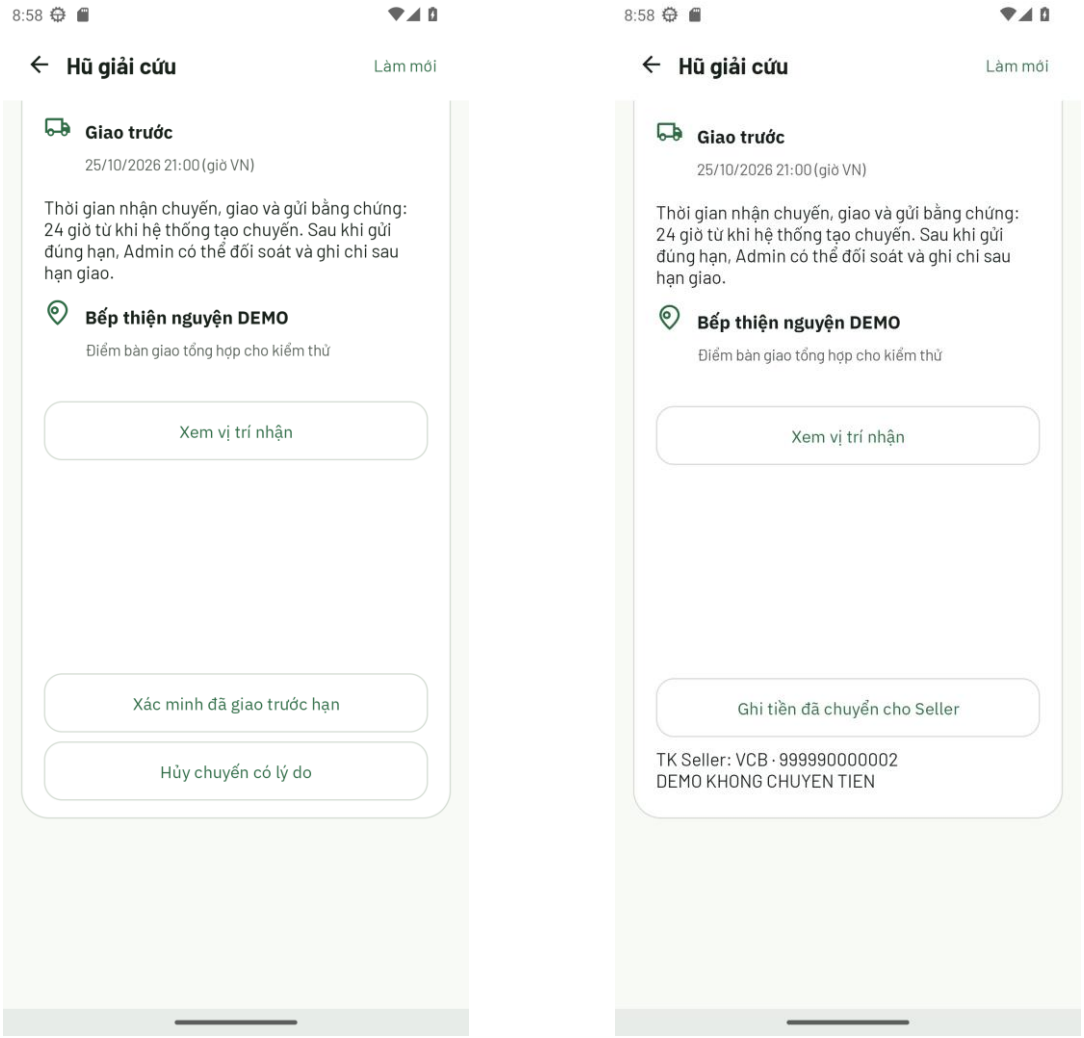
Hình 10 Hũ đã giữ tiền và chuyến Seller đã nhận với deadline giờ Việt Nam

Harness tạo hai chuyến từ donation demo 72.000đ qua webhook giả hợp lệ; mỗi chuyến giữ 36.000đ. Chuyến chính được tạo lúc 21:00 ngày 24/10/2026 giờ Việt Nam, hạn 21:00 ngày 25/10/2026. ACCEPT không gia hạn. Các giá trị này là dữ liệu test với clock được tiêm, không phải tiền hoặc một lần bàn giao ngoài đời.

3.9 Hũ chờ duyệt và ghi chi

Seller gửi bằng chứng trước deadline để chuyến chuyển DELIVERED. Khi thời điểm test đã vượt hạn, Admin vẫn được VERIFY chuyến đó; một chuyến khác chưa gửi bằng chứng bị CANCELLED và giải phóng. Điều kiện này bảo vệ Seller đã gửi đúng hạn khỏi việc bị hủy chỉ vì Admin xét muộn.

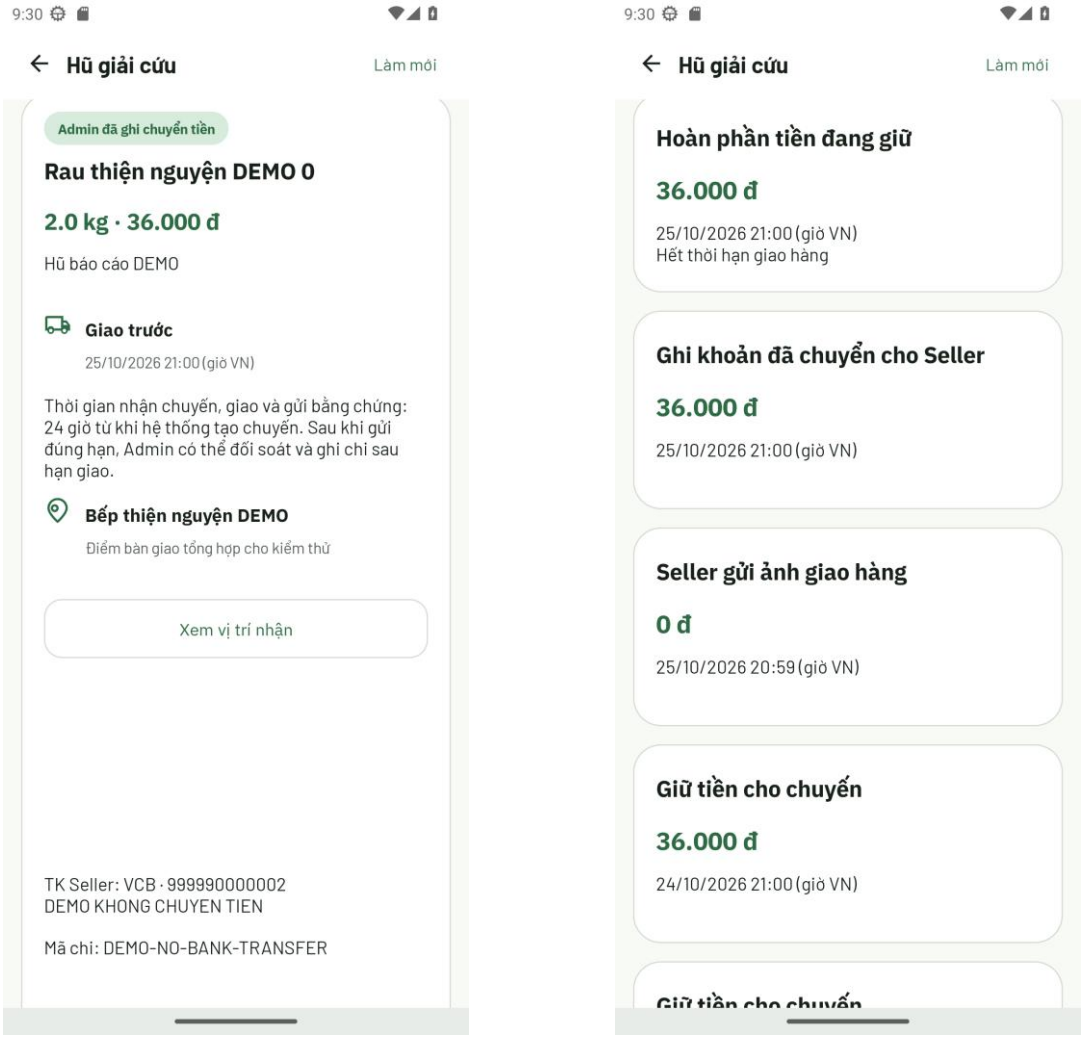
Hình 11 ghi lại giao diện đang chạy trong môi trường demo.



(a) fund-delivered-admin-jobs (b) fund-verified-admin-jobs

Hình 11 Chuyến đã gửi đúng hạn và trạng thái sau xác minh của Admin

Hình 12 ghi lại giao diện đang chạy trong môi trường demo.



(a) fund-paid-admin-status (b) fund-cancelled-admin-ledger

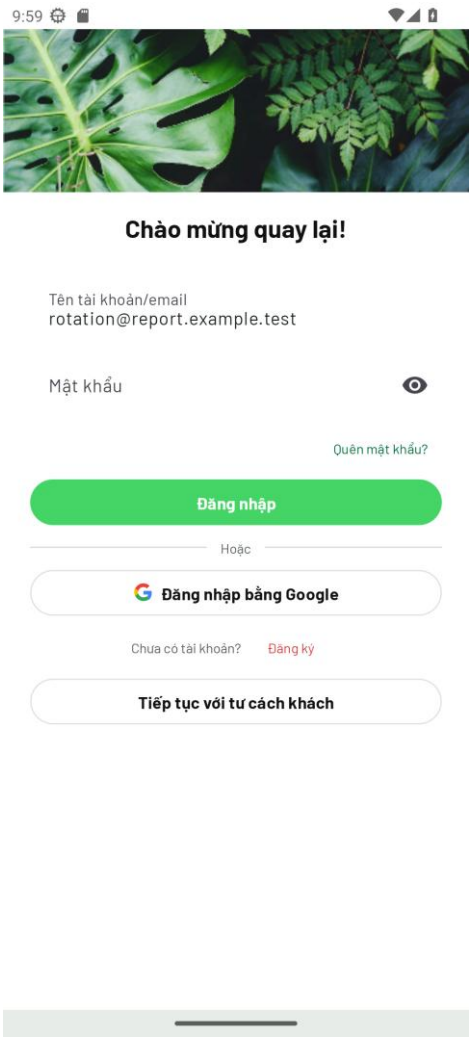
Hình 12 Trạng thái PAID của chuyến chính và ledger giải phóng chuyến quá hạn riêng

PAY kiểm tra đúng số tiền và mã giao dịch chi duy nhất. Trong bộ ảnh này, API harness thực thi transition thật, nhưng HEAD kiểm tra ảnh được stub và URL ảnh chỉ là địa chỉ test; khung ảnh có thể không tải được. Do đó các ảnh chứng minh trạng thái và chính sách tiền–kho trên emulator, chưa minh chứng chụp/upload chứng từ thật hay giao hàng và chuyển khoản thực tế. Số tiền cuối của harness được đối chiếu với ledger trong evidence.

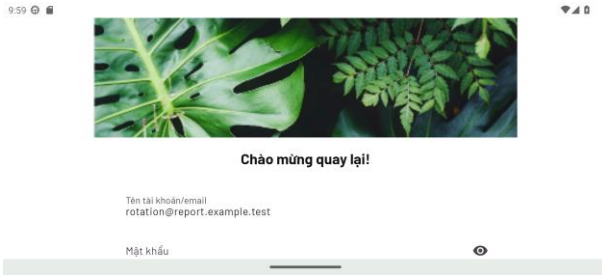
3.10 Responsive và giữ trạng thái

AndroidManifest không khóa hướng màn hình và không khai báo configChanges để tránh Activity recreation. Tài nguyên values-land, values-w600dp, layout-w600dp và layout-w840dp kết hợp các lớp ResponsiveContentLayout, ResponsiveActionLayout, ResponsiveGridLayout để giới hạn chiều rộng và sắp xếp lại nội dung. Việc có qualifier không đủ để khẳng định mọi màn đều đạt responsive [4].

Hình 13 ghi lại giao diện đang chạy trong môi trường demo.



(a) login-portrait



(b) login-landscape

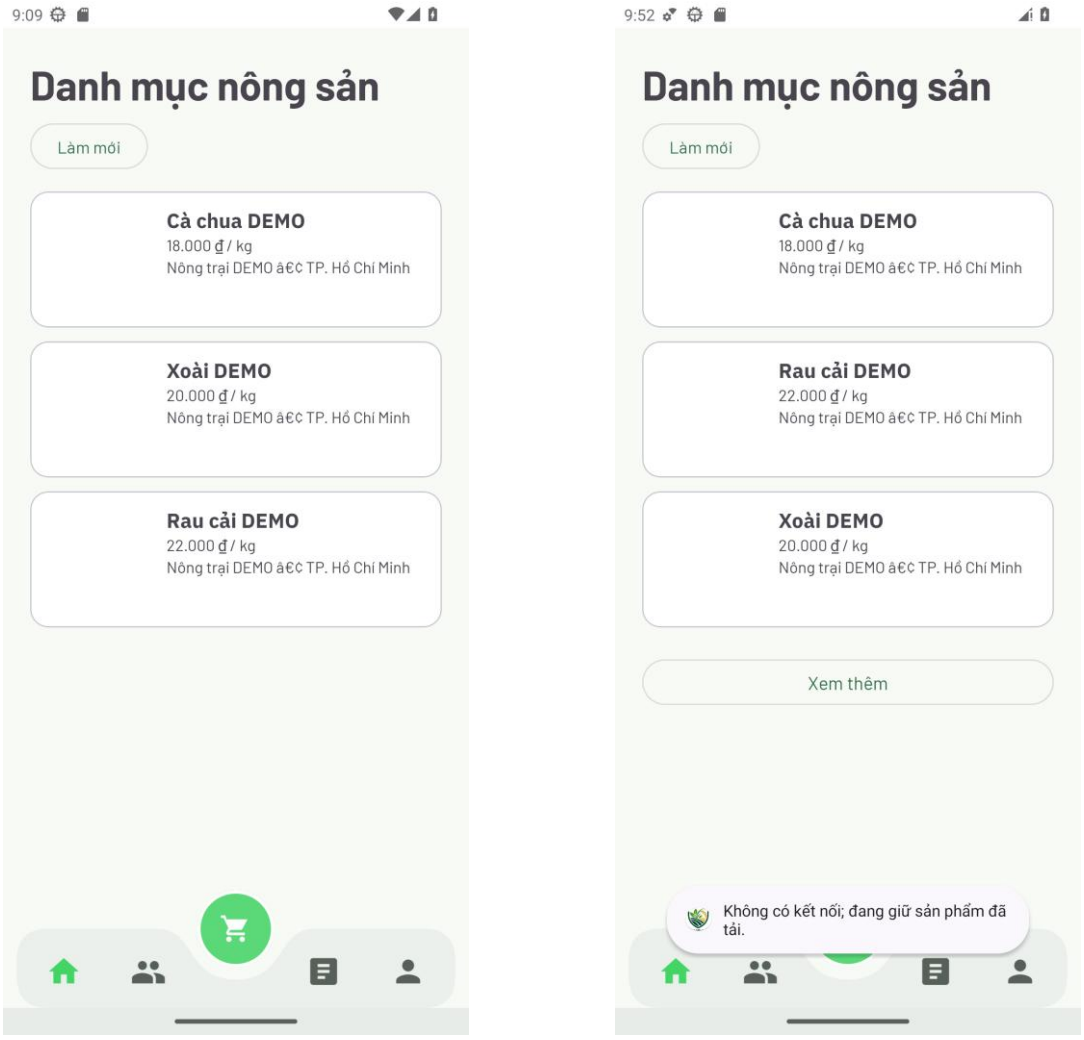
Hình 13 Cùng form đăng nhập và cùng nội dung trước và sau xoay màn

CourseRotationEvidenceTest kiểm tra form email sau xoay sang landscape và sau scenario.recreate. Trên môi trường demo đã cô lập, bài này thực sự chạy và qua, thay cho snapshot cũ từng bị skipped do phiên đã đăng nhập. ResponsiveLayoutsTest kiểm tra inflate/measure nhiều cấu hình, font 1 và 1,6 cùng hợp đồng ID; phạm vi đó bổ sung nhưng không thay thế kiểm thử thao tác tất cả Fragment, bàn phím và màn hình ngắn.

3.11 UI đọc lại danh mục khi offline

Kiểm chứng gồm hai mức. Mức đầu mở catalog online, tắt network của Firestore SDK rồi đóng/mở lại Activity; dữ liệu đã xem vẫn xuất hiện. Mức mạnh hơn tắt persistence Firestore trong APK demo, mở online để repository ghi Room, lưu cờ offline, force-stop riêng package demo rồi mở tiến trình mới. Khi đó Firestore không có disk cache và memory cache cũ; danh mục vẫn được assert có "Cà chua DEMO" trước lúc chụp. Ảnh cuối đồng thời tắt Wi-Fi/data của máy ảo test.

Hình 14 ghi lại giao diện đang chạy trong môi trường demo.



(a) catalog-online (b) catalog-room-offline-restart

Hình 14 Catalog online và dữ liệu đọc lại qua Room sau khi tiến trình khởi động lại offline

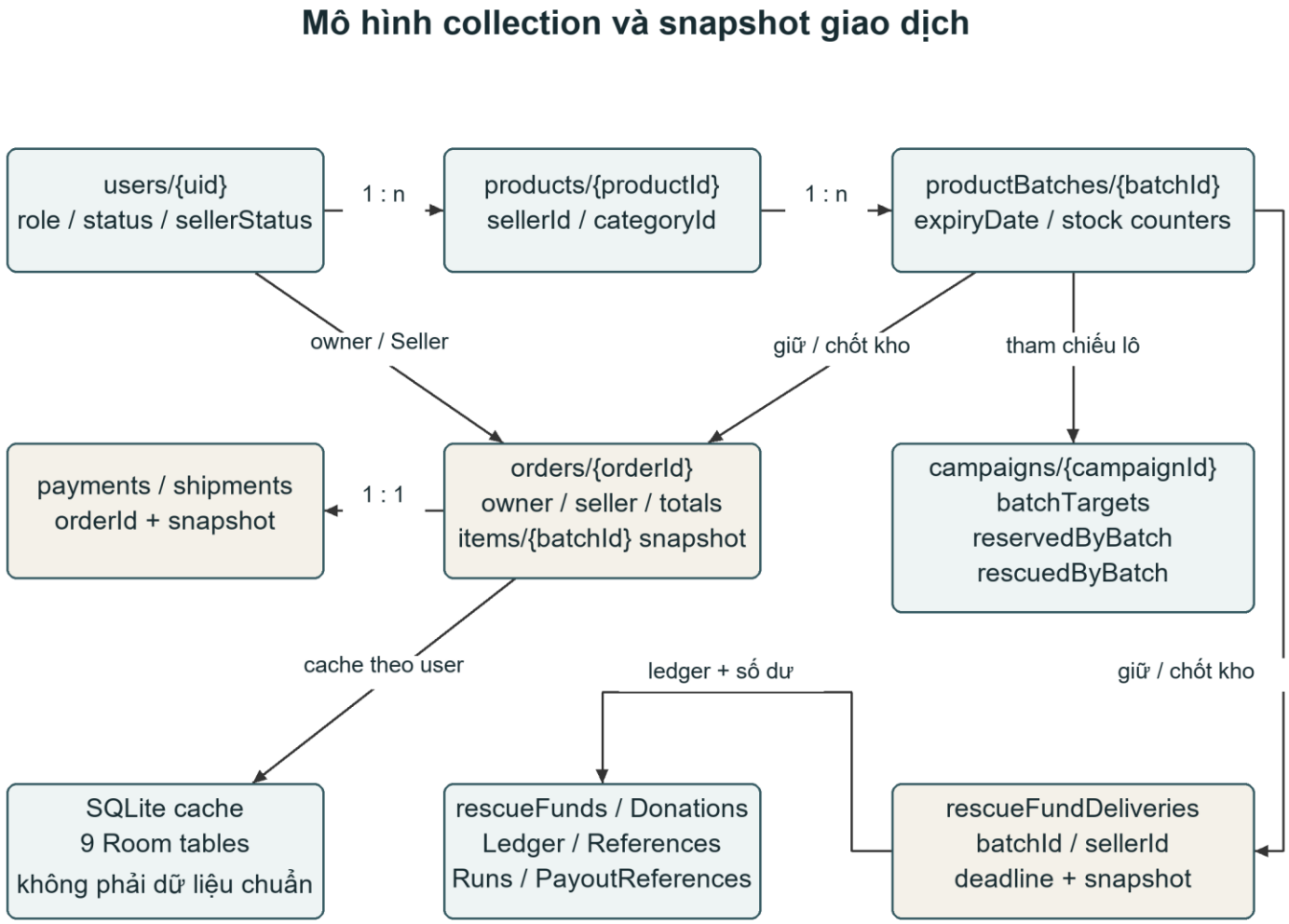
Lượt test mới chứng minh văn bản catalog đã cache còn đọc được trên UI. Không suy rộng sang mọi màn hay toàn bộ đơn của tài khoản khác. Fixture không có ảnh Product, nên chưa kiểm chứng Glide disk cache và không tuyên bố ảnh xem offline. Kết thúc test, cờ offline được xóa và network SDK được bật lại; hành động checkout vẫn phải được backend kiểm tra online.

CHƯƠNG 4 XÂY DỰNG CƠ SỞ DỮ LIỆU

4.1 Mô hình dữ liệu tổng thể

Firestore lưu collection/document và subcollection. Các trường sellerId, productId, batchId hoặc orderId là tham chiếu logic; tính đúng đắn được kiểm tra trong Rules và code transaction, không phải khóa ngoại do Firestore tự enforce. Hình 15 tập trung các quan hệ nguồn hàng và snapshot giao dịch; bảng danh mục ở các mục tiếp theo bổ sung các collection phụ trợ.

Dữ liệu chuẩn gồm hồ sơ, Product, Batch và Campaign hiện hành. Khi giao dịch được tạo, OrderItem và chuyến Hũ chụp lại các thông tin quan trọng để lịch sử không bị đổi theo tài nguyên hiện tại. Những số liệu như review summary hoặc impact là dữ liệu tổng hợp, có thể cần tác vụ đối chiếu; không nên cập nhật độc lập ở client rồi coi là nguồn tài chính.



Hình 15 Quan hệ collection, snapshot giao dịch và cache SQLite

4.2 Danh mục dữ liệu người dùng và hàng hóa

Bảng 8 tổng hợp danh mục dữ liệu người dùng và hàng hóa.

Bảng 8 Danh mục dữ liệu người dùng và hàng hóa

Collection hoặc document	Trường chủ đạo	Trách nhiệm và giới hạn
users/{uid}	role, status, sellerStatus, fullName	Hồ sơ vai trò gắn Firebase Auth UID
users/{uid}/addresses	receiverName, địa chỉ, tọa độ, isDefault	Địa chỉ riêng; scope theo chủ tài khoản
users/{uid}/devices	token thiết bị, metadata	Dùng thông báo; không đưa token vào báo cáo
sellerPublicProfiles	shopName, địa chỉ, tọa độ	Bản chiếu công khai cho storefront và khoảng cách
sellerPaymentProfiles	bankCode, tên và số tài khoản	Dữ liệu riêng, không công khai số thật
sellerApplications	nội dung đăng ký, proof, status	Hồ sơ duyệt Seller và lý do từ chối
categories	name, active, displayOrder	Danh mục hiển thị của Product
products	sellerId, categoryId, prices, unit, status	Thông tin chung, không chứa hạn của mọi lô
productBatches	productId, expiryDate, bốn bộ đếm tồn	Nguồn tồn vật lý và giữ hàng theo lô

Seller public profile và payment profile được tách để tránh dùng cùng một document cho nội dung công khai và thông tin ngân hàng. server.js có migration tách dữ liệu cũ và marker systemMigrations; việc có migration trong code không chứng minh môi trường production đã chạy. Quyền truy cập cuối cùng cần đọc cùng Rules hoặc API, không suy chỉ từ tên collection.

Nguồn kiểm tra: FirebaseUserRepository.mapActiveUser, FirebaseStorefrontRepository, FirebaseProductRepository và firestore.rules các vùng users/sellerPublicProfiles/sellerPaymentProfiles/products/productBatches.

4.3 Product và Batch cùng bất biến tồn kho

Product có originalPrice, rescuePrice, đơn vị và trạng thái hiển thị. ProductBatch có harvestDate, expiryDate, initialQuantity, availableQuantity, reservedQuantity, soldQuantity và inventoryVersion. reserveStock chuyển available sang reserved; release trả phần giữ; finalize chuyển giữ sang sold. Các thao tác cần kiểm tra số lượng hữu hạn, dương và phù hợp tồn trước khi thực hiện.

Trong phạm vi mô hình đã triển khai, initialQuantity phải cân bằng với availableQuantity cộng reservedQuantity cộng soldQuantity trong sai số cho phép. Tuy nhiên availableQuantity vật lý có thể vẫn dương khi lô đã hết hạn. getSellableQuantity đánh giá thêm thời gian và trạng thái để trả lượng được phép bán; do đó "còn hàng vật lý" và "được phép bán" là hai kết luận khác nhau.

inventoryVersion hỗ trợ phát hiện cập nhật dựa trên phiên bản cũ. Khi client gửi thao tác đã xem từ lâu, repository và backend phải đọc trạng thái hiện hành thay vì tin giá trị trên ViewModel. Hạn sử dụng không bị kéo dài chỉ để hợp thức hóa một chuyến thiện nguyện. Lô hết hạn hoặc lưu trữ giữ lịch sử và hạn chế sửa; nhập lô mới là nghiệp vụ riêng.

Bảng 9 tổng hợp product và batch cùng bất biến tồn kho.

Bảng 9 Product và Batch cùng bất biến tồn kho

Tình huống	Biến động đúng	Kết luận không được suy ra
Giữ 2 kg	available giảm 2; reserved tăng 2	Đã bán hoặc Seller đã nhận tiền
Hủy giữ	reserved giảm; available vật lý tăng	Hàng hết hạn được bán trở lại
Chốt giao hợp lệ	reserved giảm; sold tăng	Số tiền ngân hàng đã tự chi
Lô hết hạn	sellable bằng 0, giữ dữ liệu tồn	Xóa Batch và lịch sử Order

4.4 Chiến dịch và phân bổ theo lô

Bảng 10 tổng hợp chiến dịch và phân bổ theo lô.

Bảng 10 Chiến dịch và phân bổ theo lô

Dữ liệu	Mục đích	Kiểm tra liên quan
campaigns.batchTargets	Mục tiêu mỗi batchId	Lô đúng Seller, đơn vị và điều kiện Campaign
reservedByBatch	Lượng đang giữ trong từng lô	Không vượt phần mục tiêu còn lại
rescuedByBatch	Lượng đã hoàn thành từng lô	Chỉ tăng khi luồng hợp lệ chốt hàng
allocationVersion	Phiên bản bộ đếm phân bổ	Phát hiện trạng thái cũ khi sửa
startDate và endDate	Khoảng hoạt động Campaign	Đánh giá lại theo thời điểm thao tác
comments và reactions	Tương tác theo Campaign	Quyền người dùng và ngữ cảnh cha

Campaign không tự tạo tồn kho. Khi một giao dịch gắn chiến dịch giữ lô, cả bộ đếm của Batch và phần phân bổ Campaign cần được cập nhật nhất quán. Hủy cũng hoàn phần

giữ ở cả hai nơi. Với Hũ, VERIFY chuyển phần reservedByBatch sang rescuedByBatch, trong khi tiền vẫn ở trạng thái reserved chờ PAY.

Trong lựa chọn lô Hũ mới, endDate phải vượt allocatedAt cộng 24 giờ cộng một phút. Điều kiện này ngăn chọn một Campaign kết thúc quá sớm mặc dù Batch tự nó còn hạn đủ lâu. Đây là kiểm tra nghiệp vụ trong rescue-fund.js, khác với việc chỉ lọc Campaign ACTIVE ở UI.

4.5 Đơn hàng và snapshot giao dịch

Bảng 11 tổng hợp đơn hàng và snapshot giao dịch.

Bảng 11 Đơn hàng và snapshot giao dịch

Tài nguyên	Nội dung chính	Lý do tách
customerCarts/{uid}	items là mảng, selected và quantity	Giỏ hiện tại có thể đổi trước khi mua
orders/{orderId}	owner, Seller, trạng thái, tổng tiền	Giao dịch và quyền đọc theo chủ thể
orders/{orderId}/items/{itemId}	Product/Batch, tên, đơn giá, số lượng	Snapshot lịch sử tại checkout
payments/{orderId}	method, status, amount, mã tham chiếu	Tách xác minh tiền khỏi trạng thái giao
shipments/{orderId}	Thông tin nhận và giao hàng	Chỉ áp dụng phù hợp loại nhận hàng
checkoutReceipts	requestId, fingerprint, kết quả	Retry không tạo đơn hoặc giữ kho hai lần
cartMigrationReceipts	Danh tính lượt chuyển giỏ	Retry chuyển Guest cart không cộng lặp
orderReconciliationEvents	Sự kiện đối soát	Lưu lý do và chứng từ theo luồng hiện hành

Việc denormalize tên và giá vào item làm tăng dữ liệu lặp nhưng bảo vệ khả năng giải thích đơn cũ. Đổi tên Product không phải lý do viết lại item lịch sử. Ngược lại, thông tin hiện hành như tồn khả dụng phải đọc lại khi quote/checkout; không lấy snapshot trong giỏ để quyết định transaction mới.

Một số trường có tên quantityDiscount còn xuất hiện trong entity/order mapper để đọc dữ liệu cũ. Chỉ riêng tên trường không chứng minh chương trình promotion hiện còn được bán. commerce.price và chính sách hiện hành là nguồn quyết định cách tính giá cho giao dịch mới.

4.6 Dữ liệu Hũ và sổ biến động

Bảng 12 tổng hợp dữ liệu hũ và sổ biến động.

Bảng 12 Dữ liệu Hũ và sổ biến động

Collection	Nội dung	Quyền truy cập Android SDK
rescueFunds	Cấu hình và available/reserved/spent/donated	Không đọc/ghi trực tiếp; qua API
rescueFundDonations	Donation, mã, tokenHash, trạng thái	Không đọc/ghi trực tiếp
rescueFundReferences	Mã chuyển khoản tới donation	Server only
rescueFundWebhookEvents	Callback và kết quả cần đối soát	Server only
rescueFundBankTransactions	Khóa chống lặp giao dịch Hũ	Server only
rescueFundLedger	Biến động, actor, thời điểm, tham chiếu	Server only; API Admin phân trang
rescueFundDeliveries	Chuyến, snapshot, deadline, bằng chứng	API Seller theo sở hữu hoặc Admin
rescueFundRuns	Cursor ngày và hạn mức đã phân bổ	Server only; không xóa để ép chạy lại
rescueFundPayoutReferences	Mã chi đã dùng	Chặn ghi chi cùng mã cho chuyến khác

Với số dư đầu bằng không, available cộng reserved cộng spent bằng tổng donation đã xác nhận cộng tổng điều chỉnh đối soát. Ledger giải thích vì sao các số dư thay đổi. Bút toán đối soát có bằng chứng không thay donation PENDING thành PAID và không thay webhook xác nhận tiền vào. Số accumulated trong callback ngân hàng không phải số dư Hũ.

Quy tắc client cho các collection Hũ đều deny read/write, kể cả một client có vai trò Admin. API kiểm tra vai trò và trả phần dữ liệu phù hợp. Điều này tránh để app tự sửa ledger hay số dư qua SDK và làm rõ vị trí cần kiểm thử bảo mật: cả Rules và kiểm tra quyền trong module backend.

4.7 Dữ liệu giải cứu và các collection vận hành

Bảng 13 tổng hợp dữ liệu giải cứu và các collection vận hành.

Bảng 13 Dữ liệu giải cứu và các collection vận hành

Nhóm	Collection đại diện	Ý nghĩa hiện hành
Offer và claim	rescueOffers, rescueClaims	Giữ suất từ Batch, nhận tại điểm theo policy

Nhóm	Collection đại diện	Ý nghĩa hiện hành
Vị trí và tác động	rescueNearbyPreferences, rescueImpact	Vị trí chủ động và thống kê tách đơn vị
Nội dung	banners, favorites, reviews, reports	Nội dung Home và tương tác theo nghiệp vụ
Thông báo	notifications, notificationOutbox, notificationDeliveries	Inbox, hàng đợi và chống gửi trùng theo tác vụ
Media	campaignAssetReferences, campaignMediaDrafts, campaignMediaUploads	Bảo vệ tham chiếu ảnh, upload và cleanup
Ngân hàng	bankTransferReferences, sepayWebhookEvents, sepayBankTransactions	Mã đơn, callback và khóa dùng chung
Hệ thống	adminAuditLogs, systemMigrations, realtimeSubscriptions	Audit, marker và quản lý subscription

Các collection phụ trợ có thể được tạo khi xuất hiện nghiệp vụ tương ứng; không cần tạo placeholder document để "đủ sơ đồ". firestore.schema.json là tài liệu mô tả và có mục lịch sử, trong khi Rules cùng những lệnh collection trong source mới xác định đường dữ liệu hiện hành. Bản báo cáo không xóa collection thật hoặc chạy migration production để làm sạch danh mục.

Notification outbox tách việc commit nghiệp vụ khỏi gửi push. FCM có thể chậm, lỗi hoặc thiết bị offline; client mở lại màn phải lấy trạng thái có thẩm quyền. Hạn giao không bắt đầu khi người dùng nhận thông báo. Cách tách này giảm nguy cơ đem độ trễ của kênh thông báo vào bất biến tồn và tiền.

4.8 SQLite và các bảng Room thực tế

RescueFarmDatabase khai báo version 11 với chín entity. Các mapper chuyển domain object sang trường SQLite và ngược lại; URL ảnh được serialize như văn bản. Room quản lý schema và truy vấn DAO, trong khi Firestore SDK có bộ nhớ đệm riêng. Hai cơ chế có thể cùng tồn tại nhưng cần thí nghiệm phân biệt nguồn trả dữ liệu.

Bảng 14 tổng hợp sqlite và các bảng room thực tế.

Bảng 14 SQLite và các bảng Room thực tế

Bảng Room	Dữ liệu lưu	Vai trò hoặc scope
category_cache	Danh mục, active, thứ tự	Catalog công khai
product_cache	Nội dung, giá, Seller, URL ảnh, thời điểm cache	Catalog; có truy vấn theo Seller

Bảng Room	Dữ liệu lưu	Vai trò hoặc scope
product_batch_cache	Hạn, số lượng, status, inventoryVersion	Đọc khả dụng từ dữ liệu đã cache
campaign_cache	Campaign, batchTargets/counters, vị trí	Public hoặc danh sách theo Seller
banner_cache	Banner và khoảng hiển thị	Nội dung Home đã tải
guest_cart_items	Dòng giỏ Guest	Phiên Guest cục bộ; xem tên thật trong schema v11
order_cache	Header đơn và trạng thái	DAO theo ownerId hoặc sellerId
cache_metadata	scopeKey, attempt/success/expiry/error	TTL và lịch sync, scope orders:uid
cart_migration_outbox	guestId, userId, payload	Phát lại chuyển giỏ đúng đối tượng

Tên guest_cart_items được đối chiếu trực tiếp trong GuestCartItemEntity. Chín bảng này không phải bản sao đầy đủ toàn bộ Firestore; đặc biệt các collection tiền và chứng từ Hũ không được xem là cơ sở giao dịch offline. OfflineRoomEvidenceTest mở lại SQLite riêng và đọc qua OfflineProductRepository; bài UI mới bổ sung bằng chứng danh mục thật sau process restart [1,5].

4.9 TTL và đồng bộ cục bộ

Bảng 15 tổng hợp ttl và đồng bộ cục bộ.

Bảng 15 TTL và đồng bộ cục bộ

Scope	TTL trong CacheSyncPolicy	Hành vi liên quan
catalog	30 phút	Dữ liệu giá/tồn cũ cần được làm mới khi online
campaigns	10 phút	Thời gian và tồn Campaign có thể thay đổi
orders hoặc orders:uid	10 phút	Dữ liệu riêng phải ràng buộc UID
banners	1 giờ	Nội dung hiển thị có khoảng hoạt động
Hard expiry	7 ngày	Không đồng nhất TTL với quyền bán hàng

OfflineSyncScheduler dùng unique startup work và periodic work sáu giờ với điều kiện CONNECTED. CacheMetadataStore ghi lần thử, thành công và lỗi. ReadCacheSyncWorker chờ các callback trong giới hạn rồi trả success hoặc retry; một lần schedule không bảo đảm chạy tức thời hay đúng từng giây [7].

Hard expiry là chính sách code, không nên diễn giải như mọi DAO tự có điều kiện WHERE loại dữ liệu quá bảy ngày. Ví dụ CatalogCacheDao trả các hàng ACTIVE theo truy vấn hiện có; cần đọc đường gọi cụ thể để biết nơi nào kiểm tra tuổi cache. ReadCacheSyncWorker chủ động xóa order cache quá cũ. Đây là giới hạn quan trọng khi đánh giá offline thay vì chỉ chép con số TTL.

CriticalActionPolicy chỉ cho READ_SYNC vào hàng đợi, còn CHECKOUT, PAYMENT, APPROVAL, INVENTORY và ORDER_TRANSITION cần online. Giá và tồn trên cache hỗ trợ đọc lại, không đủ quyền thực hiện giao dịch. Độ tươi của dữ liệu được kiểm tra lần cuối trên backend tại transaction, không dựa vào việc TTL chưa hết.

4.10 Truy vấn tiêu biểu và index

Bảng 16 tổng hợp truy vấn tiêu biểu và index.

Bảng 16 Truy vấn tiêu biểu và index

Điểm truy vấn	Điều kiện và thứ tự thực tế	Diễn giải
publicQuery của Product	status ACTIVE, documentId, limit+1	Phân trang 12 sản phẩm mỗi lần mở rộng
Customer orders	ownerId, ownerType CUSTOMER, documentId	Không tải đơn của người khác rồi lọc ở UI
Seller orders	sellerId và con trỏ	Dữ liệu phục vụ vận hành cửa hàng
Fund ranking	status PAID, amount giảm, paidAt tăng, limit 50	Xếp hạng từng lượt, không gộp theo tên
Fund expiry	status RESERVED/ACCEPTED, deadline ≤ now	Chỉ hủy chuyến chưa gửi ảnh đúng hạn
Fund candidate	expiryDate > now+24h+1m và ≤ now+48h	Bộ lọc sơ bộ trước transaction kiểm tra lại
Room Product	SELECT ACTIVE ORDER BY name	Đọc cục bộ; không tự xác minh tồn server
Room Order	ownerId hoặc sellerId, createdAt giảm	Scope riêng trước khi phát dữ liệu UI

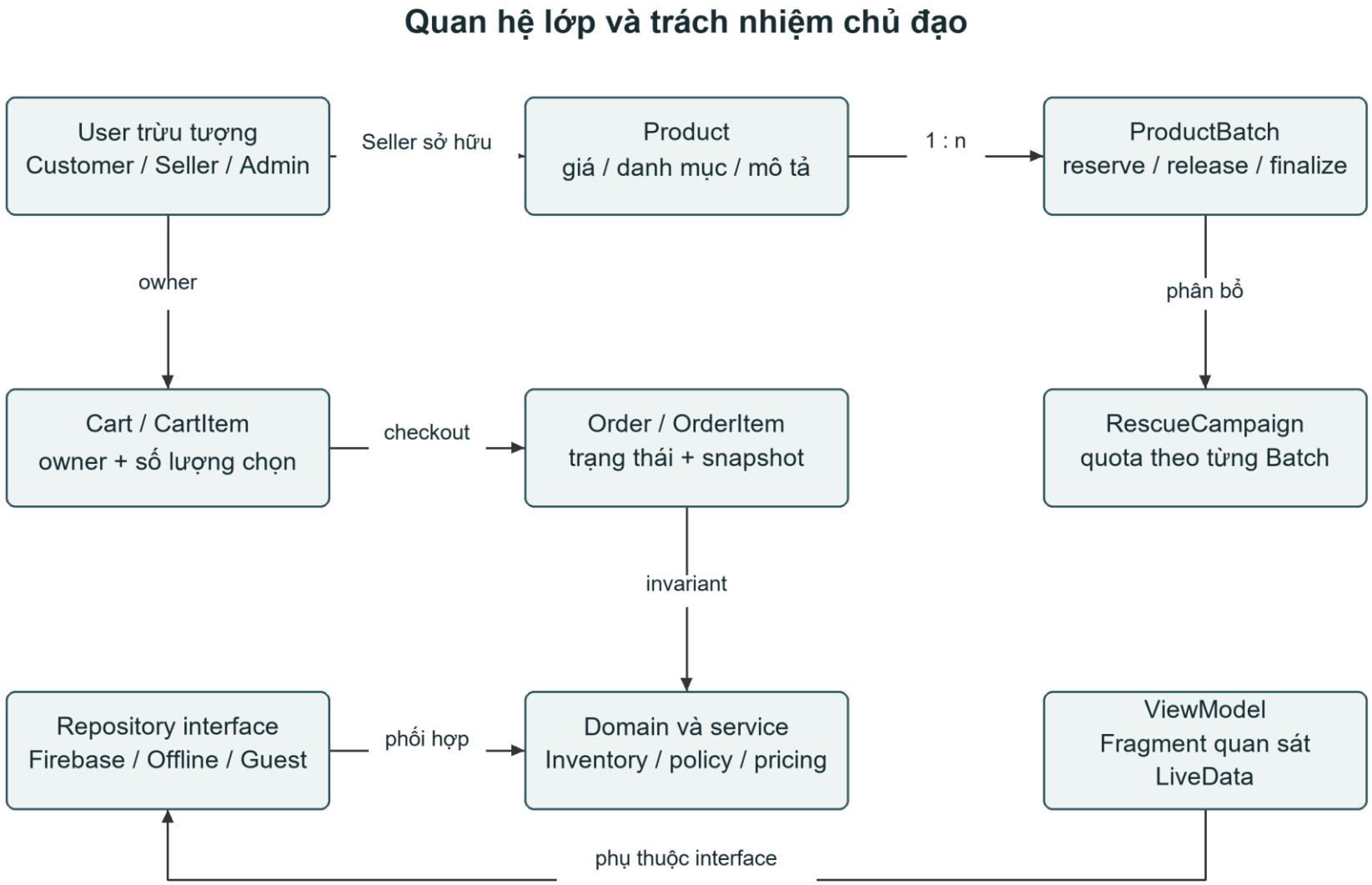
Index phải phù hợp các truy vấn trong firestore.indexes.json. Build Android hoặc test emulator thành công không chứng minh mọi composite index production đã ở trạng thái sẵn sàng. Đổi chính sách Hũ 24 giờ không thêm collection/index/Rules mới trong đợt sửa đó; một môi trường mới vẫn cần triển khai các index hiện có trước vận hành.

Không viết các truy vấn trên như SQL chạy trực tiếp ở Firestore. Chúng là mô tả query SDK/API; chỉ phần Room dùng SQL. Khi cần một trích đoạn, CatalogCacheDao.observeActiveProducts là ví dụ ngắn, đủ giải thích mục đích mà không chèn hàng trang source.

4.11 Quan hệ hướng đối tượng và mapper

Hình 16 trình bày nhóm class cốt lõi và hướng phụ thuộc. User là lớp trừu tượng với Customer, Seller và Admin; Guest được biểu diễn bằng phiên local. Product và ProductBatch tách nội dung khỏi nguồn tồn. CartItem và OrderItem có vai trò khác nhau: một bên là lựa chọn còn thay đổi, một bên là snapshot của giao dịch.

Repository interface giúp ViewModel không phụ thuộc trực tiếp toàn bộ chi tiết Firebase hay SQLite. CatalogCacheMapper, CampaignCacheMapper và OrderCacheMapper phục hồi domain object từ cache. Test mapper chỉ xác nhận chuyển đổi trường và trường hợp biên đã viết; nó không tự chứng minh người dùng đã hoàn thành checkout, nhận FCM hoặc tải được ảnh khi offline.



Hình 16 Quan hệ lớp rút gọn và trách nhiệm của các tầng

CHƯƠNG 5 XÂY DỰNG ỨNG DỤNG

5.1 Cấu trúc mã và môi trường build

Bảng 17 tổng hợp cấu trúc mã và môi trường build.

Bảng 17 Cấu trúc mã và môi trường build

Khu vực	Trách nhiệm	Điểm cần hiểu khi bảo trì
ui/	Fragment, ViewModel, state, factory	Tránh đưa bất biến tài chính vào xử lý View
domain/	Model, enum và quy tắc nghiệp vụ	Phân biệt quantity vật lý và khả dụng
data/repository/	Hợp đồng và lựa chọn nguồn	Guest, offline và Firebase có khác biệt
data/remote/firebase/	SDK query, mapper, HTTP bridge	Theo dõi lifecycle listener và quyền
data/local/	Room entity, DAO, mapper, migration	Không coi URL là file ảnh
service/	Auth, media, location, sync, notification	Các provider cần xác minh runtime riêng
functions/	Express và module tin cậy	Admin SDK cần tự kiểm tra role/owner
cloudinary-worker/	Ký upload và bảo vệ asset	Secret ở server, không nhúng Android

app/build.gradle.kts xác định compile/target SDK 37, min SDK 29, Java source/target 11. Phiên bản thư viện lấy từ gradle/libs.versions.toml. gradle-daemon-jvm.properties yêu cầu toolchain 25 cho daemon, trong khi CI còn cấu hình JDK khởi chạy riêng; không đồng nhất phiên bản JDK chạy Gradle với mức Java của source hoặc Android API thiết bị.

APK demo phục vụ báo cáo là một build type mới có applicationIdSuffix .reportdemo và endpoint local. Android release/debug mặc định vẫn giữ cấu hình hiện hành. Những bổ sung này chưa commit; report evidence lưu HEAD nền cùng diff và SHA-256 APK, thay vì giả định APK trùng byte với một bản release đã có trên GitHub.

5.2 Kết nối Firebase và phân quyền

FirebaseConfiguration kiểm tra tài nguyên google_app_id để repository biết cấu hình Firebase đã có hay chưa. App thường khởi tạo Firebase và cài App Check provider tương ứng build type. Bản reportDemo thay Application riêng, kiểm tra projectId bắt buộc bằng demo-rescuefarm trước khi gắn useEmulator cho Auth và Firestore. HTTP backend được override độc lập; chỉ gắn Firestore Emulator ở backend không tự chuyển Android sang emulator.

Rules kiểm tra dữ liệu client bằng trạng thái tài khoản, vai trò, sở hữu và tập trường được phép đổi. Dữ liệu server-only bị deny. server.js xác minh ID token và document users đang ACTIVE; router tiếp tục kiểm tra quyền cho thao tác cụ thể. Chính vì Admin SDK là đường tin cậy, test client Rules và test API là hai nhóm bằng chứng khác nhau.

Đối với Hũ, cả Admin client cũng không sửa trực tiếp rescueFunds hay ledger qua SDK. Giao diện gửi request vào API; backend dùng transaction và tạo ledger. Với dữ liệu public, query cần phù hợp Rules, không phải tải mọi document rồi chỉ ẩn hàng không được xem ở Java.

Nguồn: RescueFarmApplication, AppCheckInstaller theo build type, FirebaseConfiguration, firestore.rules và functions/server.js. Demo bỏ provider App Check ngoài và dùng danh tính giả lập; không chứng minh Play Integrity hoặc App Check production đã được kiểm chứng.

5.3 Xác thực và Google Credential Manager

Luồng email dùng FirebaseAuthRepository và FirebaseUserRepository để lấy hồ sơ sau xác thực. AuthViewModel quản lý trạng thái IDLE/LOADING/ERROR và vai trò. Lỗi đăng nhập được chuyển thành thông báo phù hợp; hồ sơ bị khóa khác với lỗi mạng hoặc thông tin đăng nhập không đúng. Vai trò Admin không phải lựa chọn tự đăng ký của người dùng mới.

GoogleCredentialService sử dụng CredentialManager, GetSignInWithGoogleOption và GoogleIdTokenCredential, sau đó FirebaseAuthRepository trao đổi token với Firebase Auth. Code tìm tài nguyên default_web_client_id và báo lỗi rõ khi thiếu. Yêu cầu scope trong luồng hiện tại là đăng nhập danh tính; không có cơ sở suy ra quyền đọc Gmail, Drive hoặc dữ liệu Google khác [6].

Trong lượt báo cáo, tài khoản demo đăng nhập bằng Auth Emulator; không chạy màn chọn tài khoản Google thật. Vì vậy kết luận là có triển khai Google Sign-In, chưa xác minh tích hợp provider ngoài trên cấu hình SHA/OAuth hiện hành. Bộ ảnh form đăng nhập không được dùng để thay thế bằng chứng OAuth thành công.

Bảng 18 tổng hợp xác thực và google credential manager.

Bảng 18 Xác thực và Google Credential Manager

Lớp	Vai trò	Kết quả cần nhận
LoginFragment	Nhập liệu và sự kiện	Gửi yêu cầu; hiển thị trạng thái
AuthViewModel	Điều phối phiên	User/role hoặc lỗi có nghĩa
GoogleCredentialService	Credential Manager	ID token hoặc lý do thất bại
FirebaseAuthRepository	Firebase Auth	Phiên Firebase hợp lệ

Lớp	Vai trò	Kết quả cần nhận
FirebaseUserRepository	Hồ sơ nghiệp vụ	User ACTIVE và role phù hợp

5.4 Triển khai Home và tìm kiếm

HomeViewModel phối hợp dữ liệu catalog, lô và chiến dịch rồi tạo HomeViewState để Fragment dựng nội dung. HomeContentBuilder và HomeCardRenderer hỗ trợ trình bày các nhóm card. Việc tách state giúp màn hình không phải tự quyết định tất cả quy tắc khả dụng ở từng TextView. Với danh sách phân trang, remote limit và trạng thái còn dữ liệu được quản lý trong repository.

DiscoveryEngine nhận dữ liệu và tiêu chí để trả kết quả, thuận lợi cho unit test không cần thiết bị. Search vẫn phải gắn lô ứng viên vì một Product ACTIVE có thể không còn Batch bán được. Các điều kiện về bán kính cần tọa độ đủ tin cậy; AndroidLocationProvider xử lý vị trí và có logic loại tọa độ mặc định emulator để không âm thầm thay vị trí đã lưu.

Màn lọc và lựa chọn số lượng hiện có được giữ nguyên trong lượt báo cáo. Không sửa thiết kế header, logo hoặc màu để làm ảnh đẹp hơn. Một màn có dữ liệu thiếu sẽ được ghi giới hạn thay vì thêm text giả chỉ dành cho ảnh. Regression search đã có trong suite JVM; ảnh minh chứng chỉ bổ sung việc UI chạy với fixture được ghi nguồn.

5.5 Triển khai chi tiết sản phẩm và giỏ

ProductViewModel phối hợp ProductRepository với domain Product/Batch. ProductDetailFragment dùng trạng thái hiện hành để hiển thị giá và tồn, nhận batchId hoặc ngữ cảnh Campaign từ arguments. Khi nhập số lượng, UI kiểm tra tính hợp lệ và định dạng đơn vị; backend vẫn kiểm tra lại. Giá cơ bản lấy rescuePrice hiện hành, không phục hồi promotion hoặc wallet cũ.

GuestCartRepository dùng GuestCartDao để giữ các dòng cục bộ. FirebaseCartRepository đọc/ghi customerCarts gắn UID đang đăng nhập và theo dõi thay đổi Auth. Chuyển giỏ Guest dùng outbox cục bộ cùng CartMigrationService/Worker và receipt phía server. Một retry phải nhận lại kết quả hoặc bỏ qua phần đã áp dụng, tránh cộng quantity nhiều lần khi app bị đóng giữa chừng.

CartFragment cần dữ liệu Product để trình bày tên/giá bên cạnh dòng giỏ. Việc một item còn trong Room không chứng minh lô vẫn bán được. Khi revalidate không thành công, UI cần giữ thông tin đủ để người dùng hiểu vấn đề, đồng thời chặn việc coi giá/tồn cũ là một giao dịch đã được giữ.

Nguồn: ui/product/, ui/cart/, data/repository/GuestCartRepository.java, FirebaseCartRepository.java và functions/cart-migration.js. Bằng chứng runtime là click

thêm hàng Guest và giỏ Customer trong screenshots-run; logic retry được chứng minh riêng trong backend-unit và suite Firestore/API.

5.6 Triển khai checkout và xác nhận tiền

CheckoutRequestIdentity lưu requestId qua SavedStateHandle. CheckoutViewModelFactory dùng CreationExtras để tạo SavedStateHandle tương thích vòng đời; CheckoutFragment còn lưu các trường chọn địa chỉ và trạng thái form qua Bundle. Những cơ chế này giảm nguy cơ một lần recreate sinh danh tính checkout mới, nhưng vẫn cần server receipt để ngăn trùng trong những tình huống retry khác.

commerce.js phân biệt bước quote và bước transaction. Trong transaction, dữ liệu hiện hành được đọc trước khi ghi giữ kho, Order/Item và receipt. Khi có cạnh tranh, logic được đánh giá lại với snapshot mới; không ghi một số dư bằng phép cộng từ UI. Đây là lý do code kiểm tra đủ tiền/tồn ở server dù màn hình đã có xác thực đầu vào [2].

payment-qr.js và sepay.js tạo/đọc dữ liệu QR và tiếp nhận callback. verifyWebhook xác thực theo cấu hình merchant; mapping mã xác định đơn hoặc Hũ; khóa giao dịch ngăn dùng lại cùng lần chuyển. Một request "tôi đã chuyển tiền" từ điện thoại không thay callback ngân hàng. Probe id bằng 0 không được cộng tiền. Cơ chế xác thực webhook được đối chiếu tài liệu SePay; kết quả vận hành RescueFarm phải dựa vào test và môi trường cấu hình thực tế [8].

Những gì đã kiểm thử là logic module, Rules và API trên emulator với payload giả; không có thanh toán ngân hàng thật. Đăng ký webhook production, mã ngân hàng và kết nối Render cần cấu hình hợp lệ ngoài môi trường demo trước khi khẳng định tích hợp live.

5.7 Triển khai Seller và vòng đời chiến dịch

Seller thao tác Product, Batch, Campaign, Order và các chương trình giải cứu từ mobile. Factory tạo repository tương ứng, ViewModel giữ trạng thái và Fragment điều khiển form. Những hành động ảnh hưởng kho yêu cầu kiểm tra version và nguồn dữ liệu hiện hành. Từ chối cập nhật lô đã hết hạn hoặc đã có biến động không phải lỗi hiển thị; đó là ràng buộc bảo vệ lịch sử.

campaign-inventory.js quản lý phần mục tiêu và bộ đếm theo Batch. campaign-lifecycle.js đối chiếu chiến dịch theo thời gian; batch-expiry.js xử lý hậu quả lô quá hạn cùng các đơn liên quan. Cách phối hợp này quan trọng vì một Campaign có thể vẫn có document ACTIVE trong một khoảng trễ tác vụ nền, nhưng giao dịch mới vẫn phải kiểm tra điều kiện thời gian đồng bộ.

Seller order UI trình bày các đơn của cửa hàng, còn seller-analytics.js tổng hợp theo các phạm vi được hỗ trợ. Một chỉ số dashboard không phải phép đo hiệu năng hoặc tác động môi trường. Khi chưa có fixture đầy đủ cho Campaign/Offer, báo cáo chỉ ghi có code và đã qua test module, không nâng lên mức đã demo tất cả chức năng trên thiết bị.

5.8 Triển khai tương tác và quản trị

FirebaseEngagementRepository cùng ReviewEligibilityService kiểm tra điều kiện review gắn mua hàng; campaign engagement xử lý comment và reaction trong đúng Campaign. Report có loại và mục tiêu để Admin đọc theo ngữ cảnh. Trạng thái xóa/ẩn hay kết quả moderation cần giữ ý nghĩa lịch sử và phản hồi cho người dùng thay vì chỉ xóa khỏi danh sách không có giải thích.

AdminRepository định nghĩa Section và các DTO cho dashboard/list/detail. FirebaseAdminRepository phối hợp query và API tin cậy. AdminDetailRenderer chuyển dữ liệu thành phần trình bày thích hợp trên mobile. Một số công việc quản trị có thể cần ghi lý do hoặc chứng từ; quyền Admin không miễn các kiểm tra invariant tiền/kho.

Mục tiêu demo quản trị là chọn một thao tác đại diện có dữ liệu sẵn và giải thích đường quyền, không trình diễn toàn bộ backend qua nhiều màn rỗng. Trong bộ ảnh, Admin đọc dashboard và Product list. Chưa thao tác phê duyệt Seller đầu cuối trong lượt chụp; các test Rules/API mới là bằng chứng kiểm tra quyền và mutation theo ca đã viết.

5.9 Triển khai Hũ từ tiền vào tới khoản chi

rescue-fund.js nhận dependency db, admin, authenticate, env, now và fetchImpl. Cấu trúc này cho phép test thời gian và ảnh mà không đợi 24 giờ hoặc dùng provider thật. Donation giữ tokenHash; biên nhận yêu cầu token bí mật của thiết bị. Bảng xếp hạng public chỉ lấy donation PAID và loại trường riêng, không gộp người theo tên hiển thị.

allocation đọc chuyến có khóa ngày–lô, quỹ, lượt chạy, Batch, Product, Seller, payment profile và các giao dịch thành công gần đây. Giá trị còn phân bổ là phần nhỏ nhất giữa tiền khả dụng, ngân sách ngày và ngân sách Seller. Đơn vị đếm lấy số nguyên; khối lượng/thể tích cho ba chữ số thập phân; số tiền được làm tròn lên và kiểm tra không vượt ngân sách. Transaction cập nhật tiền, kho, Campaign counters, ledger và notification thích hợp.

action phân biệt ACCEPT, DELIVER, VERIFY, PAY và CANCEL. DELIVER kiểm tra deadline và ảnh; VERIFY chốt kho; PAY chuyển reserved sang spent sau chứng từ; CANCEL giải phóng đúng phần giữ. Retry đối chiếu trạng thái và dữ liệu định danh, không cộng/trừ thêm. Chuyến cũ không bị viết lại deadline khi policy mới được áp dụng.

RescueFundFragment dùng RenderBackendClient, không ghi trực tiếp collection Hũ. Poll receipt khi màn đang mở dùng chu kỳ 30 giây ở HEAD này; backend public cache 30 giây và receipt cache 10 giây là các cơ chế khác nhau. Độ trễ webhook có thể làm thời điểm ledger khác lúc người dùng chuyển tiền; đó không phải lý do sửa timestamp lưu trữ.

5.10 API vị trí và media

AndroidLocationProvider phối hợp Fused Location Provider, GPS đã có và Geocoder theo quyền được cấp. CampaignLocationService tính vị trí phù hợp của chiến dịch và khoảng cách. Bấm xem nơi nhận mở geo intent; không có bằng chứng cho Google Maps SDK nhúng hoặc theo dõi giao hàng liên tục. Ngoài emulator, cần kiểm tra riêng cấp quyền, vị trí yếu và địa chỉ thực tế.

CloudinaryMediaStorage lấy chữ ký hoặc gọi endpoint upload Campaign có bảo vệ từ Worker. Worker xác minh token và hồ sơ, ràng buộc asset theo chủ thể/resource và kiểm tra tham chiếu được bảo vệ trước xóa hoặc ghi đè. Cloudinary API secret chỉ ở môi trường server; Android giữ cloud name và URL cấu hình, không được nhúng khóa bí mật.

Trong ảnh Hũ demo, fetchImpl được stub để test transition và không gửi HEAD thật ra Cloudinary. Điều đó không chứng minh file đã upload hoặc ảnh có sẵn khi offline. Để demo camera/chứng từ hoàn chỉnh cần một môi trường media test hợp lệ và dữ liệu ảnh tổng hợp; không dùng tài khoản ngân hàng hoặc chứng từ cá nhân để lấp chỗ trống.

Bảng 19 tổng hợp api vị trí và media.

Bảng 19 API vị trí và media

API hoặc dịch vụ	Có code	Xác minh trong lượt này
Firebase Auth/Firestore	Có	Emulator và Rules/API
Google Sign-In	Có	Chưa provider OAuth thật
Location/Geocoder	Có	Màn demo dùng tọa độ tổng hợp; chưa thực địa
Cloudinary/Worker	Có	9 test Worker; chưa upload ngoài
Render HTTP	Có	Logic server local; chưa deploy live
SePay/VietQR	Có	Webhook giả và idempotency; chưa tiền thật
FCM	Có	Outbox test; chặn gửi push thật trong harness

5.11 Đồng bộ và xử lý lỗi

SnapshotLiveData sở hữu listener theo vòng đời và nhận LiveData Room làm fallback. Một snapshot local rỗng chưa đủ chứng minh server đã xóa dữ liệu, nên code không thay nguồn Room bằng kết quả rỗng chưa hoàn chỉnh. Khi permission denied, nguồn không được tiếp tục phát dữ liệu riêng như trước; khi owner thay đổi, listener cũ bị tháo và kết quả được làm rỗng.

ReadCacheSyncWorker gọi các refresh theo catalog/campaign/banner/order; scope order lấy UID và role hiện hành. Kết quả thất bại hoặc timeout được ghi vào metadata và trả retry. Unique work hạn chế chồng tác vụ. Việc dùng WorkManager không có nghĩa worker luôn chạy chính xác mỗi sáu giờ hoặc luôn hoàn thành khi người dùng tắt app [7].

Ở backend, read-cache và quota guard giảm đọc lặp trong những đường được hỗ trợ và trả trạng thái tạm dừng khi gặp quota. Lỗi dịch vụ cần phân biệt với dữ liệu rỗng. Những cơ chế này có code và test logic, nhưng chưa có phép đo latency, tải đồng thời hoặc chi phí production để đưa con số vào báo cáo.

5.12 Tích hợp Git và CI

Repository có GitHub remotes, nhiều merge commit và workflow android-package.yml. Pipeline kiểm tra navigation, backend check/unit, suite Firestore và đóng APK theo cấu hình. Gói APK được phân phối qua Actions artifact hoặc release khi phù hợp; đây không phải bằng chứng một bản đã được phát hành lên Play Store hay backend Render đã deploy cùng revision.

Các merge gần HEAD gồm PR #197 tại 28adb48 cho policy Hũ; PR #196 tại 3e59e15 cho responsive; PR #195 tại bf648f2 cho thay đổi UI/search; PR #192 tại 3e80abd từ nhánh nguyentankhiem1610/main. Tên branch trong merge chứng minh lịch sử tích hợp; không tự chứng minh tác giả duy nhất của mọi dòng trong module.

Lượt báo cáo giữ nguyên thay đổi có sẵn của AI_REPORT_PROMPT_VI.md, không reset/stash/checkout đè và không commit/push/deploy. Phần bổ sung phục vụ evidence được ghi trong handoff cùng diff. Nhóm cần xem lại và quyết định đưa build type demo vào repository hay giữ như công cụ thực nghiệm; mặc định release không bị chuyển sang emulator.

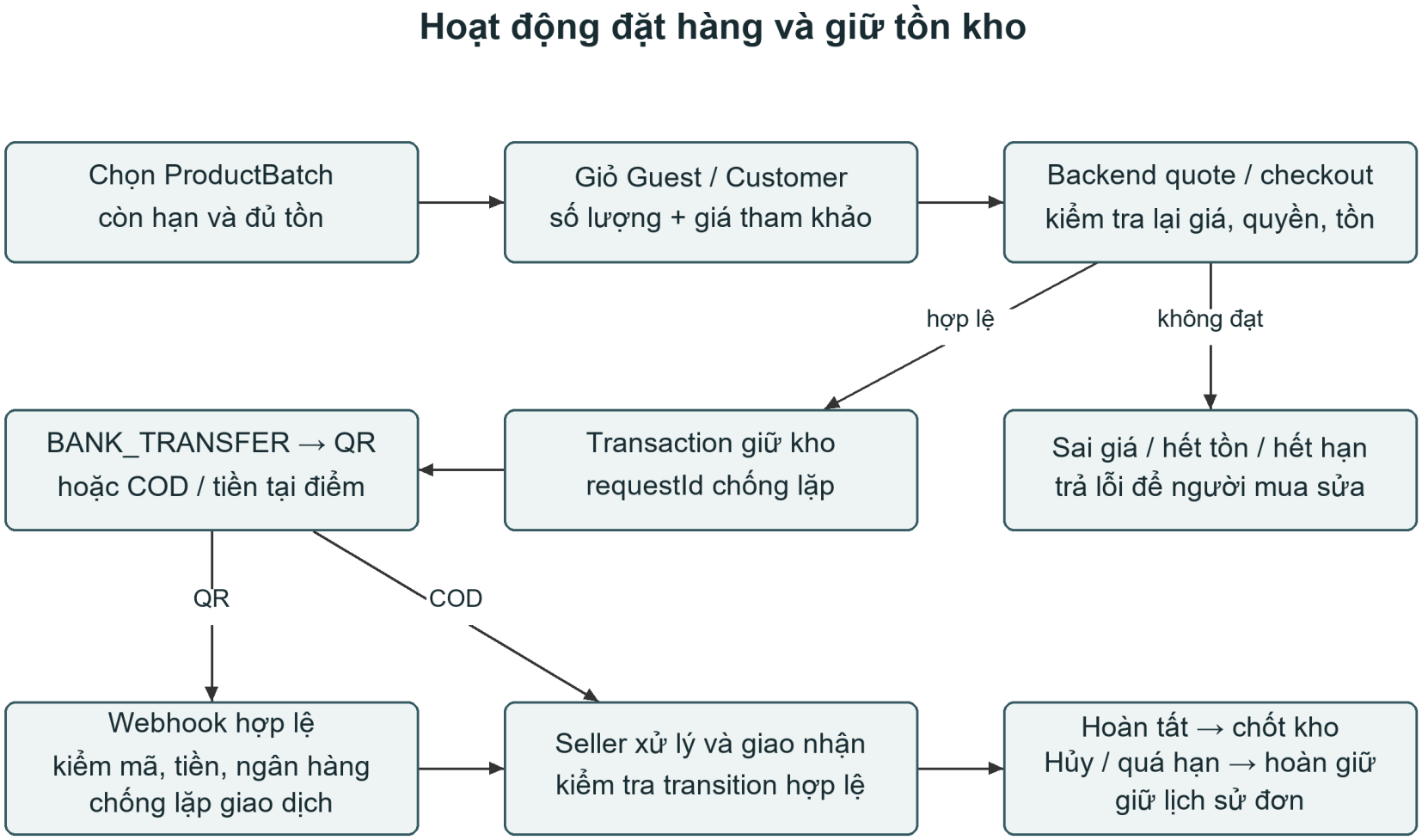
CHƯƠNG 6 QUY TRÌNH THỰC THI VÀ KIỂM THỬ

6.1 Quy trình đặt hàng tổng hợp

Hình 17 đọc từ trái sang phải ở hàng đầu, sau đó theo nhánh hợp lệ hoặc lỗi. Người mua chọn Batch; giỏ lưu lựa chọn; backend quote và checkout kiểm tra lại. Nhánh thất bại trả lỗi để sửa lựa chọn. Nhánh thành công giữ kho trong transaction trước khi đi qua phương thức thanh toán và giao nhận.

Với QR, callback ngân hàng hợp lệ xác nhận tiền vào và phải chịu kiểm tra chống lặp. Với COD, quá trình nhận tiền theo chính sách khác; không giả một callback QR để đẩy đơn sang đã trả. Khi giao/hủy/quá hạn, code kiểm tra transition rồi chốt hoặc giải phóng phần giữ. Nhánh ngoại lệ được xem là thành phần thiết kế, không phải những bước được bỏ qua để demo nhanh.

Nguồn chính: functions/commerce.js, sepay.js, campaign-lifecycle.js, batch-expiry.js và FirebaseOrderRepository. Thực nghiệm trong lượt này gồm suite API/Rules; màn checkout và danh sách đơn đã mở nhưng chưa có ảnh QR đơn hoàn chỉnh hoặc review sau giao nhận qua UI.



Client không tự đánh dấu đã nhận tiền. Webhook retry không tạo thêm khoản thu.
Đơn giao nhận và trạng thái thanh toán là hai chiều trạng thái cần đối chiếu.

Hình 17 Luồng đặt hàng với kiểm tra tồn, QR hoặc COD và nhánh lỗi

6.2 Quy trình tài khoản và thêm sản phẩm

Đăng ký bắt đầu bằng vai trò được phép, xác thực Firebase và tạo hồ sơ nghiệp vụ. Customer có thể sử dụng chức năng mua sau khi hồ sơ hợp lệ; Seller cần hồ sơ đăng ký và sự phê duyệt. Không có luồng tự tạo Admin qua lựa chọn ở form. Đăng nhập khôi phục vai trò từ hồ sơ và cập nhật khi trạng thái tài khoản thay đổi.

Thêm sản phẩm cần danh mục, tên, đơn vị và giá hợp lệ; ảnh đi qua media signing khi sử dụng. Sau đó Seller tạo Batch với ngày thu hoạch, hạn sử dụng và lượng đầu. Tách hai bước giúp một Product được nhập nhiều đợt hàng với hạn riêng. Gắn Campaign phải tham chiếu các lô thuộc Seller và phù hợp đơn vị, không lấy một ID bất kỳ chỉ vì UI cho nhập.

Bảng 20 tổng hợp quy trình tài khoản và thêm sản phẩm.

Bảng 20 Quy trình tài khoản và thêm sản phẩm

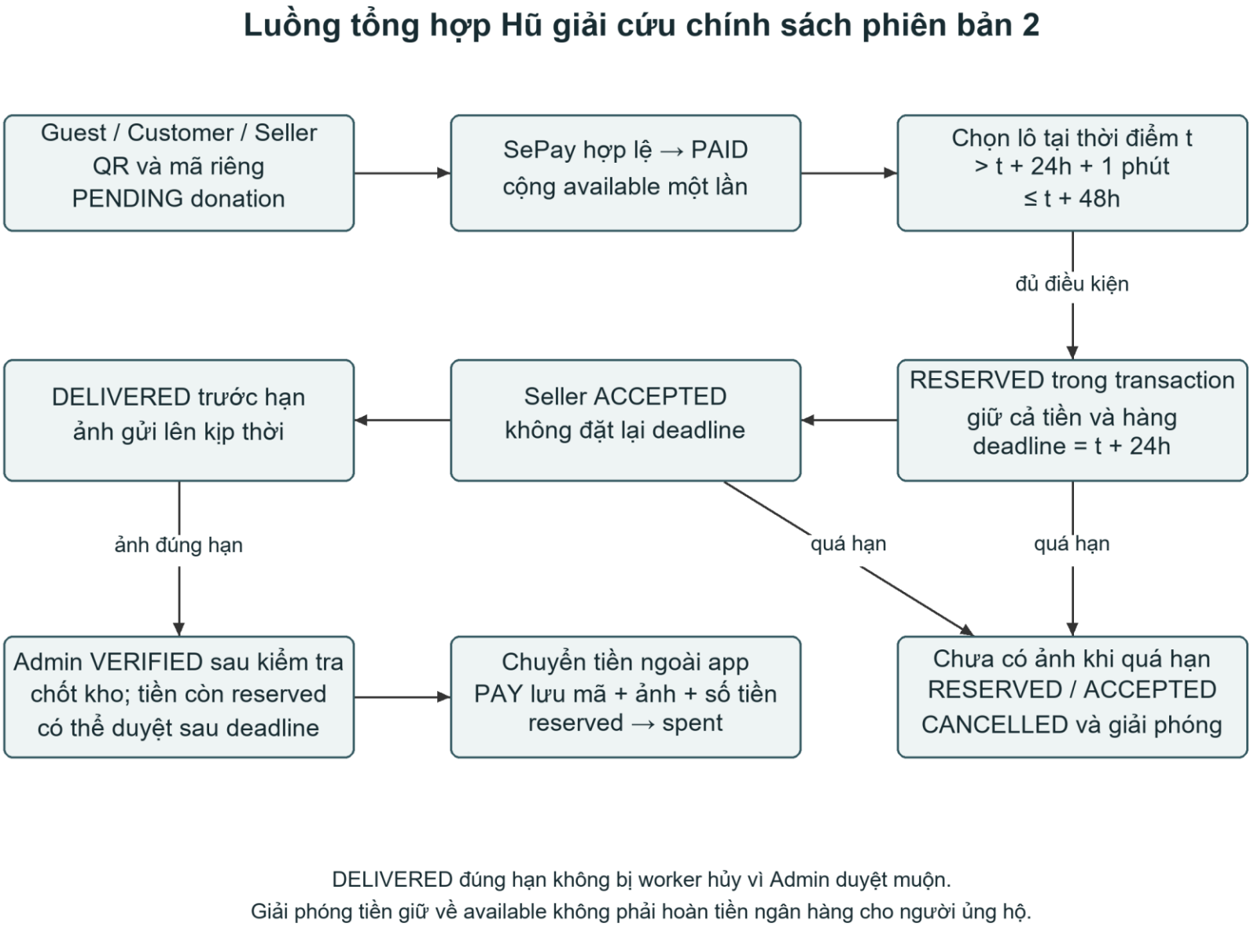
Quy trình	Kết quả lưu	Ngoại lệ quan trọng
Đăng ký Customer	Auth + users profile	Email trùng, mật khẩu yếu, lỗi tạo hồ sơ
Đăng ký Seller	Seller application và trạng thái duyệt	Hồ sơ thiếu, bị từ chối, tài khoản khóa
Đăng nhập	Firebase session và User ACTIVE	Sai thông tin, mất mạng, hồ sơ không khả dụng
Tạo Product	products có Seller và giá hợp lệ	Giá sai, category thiếu, lỗi upload
Nhập Batch	Hạn và số lượng theo lô	Ngày sai, lượng không dương, quyền không đúng

Các dòng này mô tả code hiện hành và điều kiện cần test. Bộ ảnh đã có xác thực tài khoản seed và danh sách Seller; chưa đại diện cho toàn bộ form đăng ký, phê duyệt hoặc upload mới. Hạn chế đó được giữ nhất quán với ma trận audit.

6.3 Quy trình Hũ và thí nghiệm thời gian

Hình 18 gom tiền vào, chọn hàng, 24 giờ của Seller và bước duyệt/chi của Admin trên cùng canvas. Lượt phân bổ theo ngày Việt Nam thường được đánh thức quanh 00:00; cron, Render ngủ hoặc khởi động muộn có thể làm thời điểm thực tế trễ. Deadline tính từ allocatedAt thực tế nên chuyến tạo buổi tối vẫn có đủ 24 giờ.

Harness fund-demo.cjs tạo quỹ bằng admin/save với số dư đầu bằng không, tạo donation, gửi webhook giả hợp lệ và replay, sau đó gọi module allocation cho hai lô fixture. Chuyến chính đi qua ACCEPT, DELIVER sát hạn, VERIFY sau hạn và PAY idempotent. Chuyến thứ hai không gửi ảnh, được reconcile hủy sau hạn. Không gán thẳng status PAID hoặc sửa số dư để tạo ảnh.



Hình 18 Hoạt động Hũ với nhánh quá hạn và duyệt sau hạn

Trong harness local, 72.000đ donation demo kết thúc thành 36.000đ available, 0đ reserved và 36.000đ spent. Clock được tiêm; HEAD ảnh được stub. Không có giao hàng, upload hoặc chuyển khoản thật. Mỗi phase lưu JSON/ledger và assert cân bằng, không gán thẳng status hoặc số dư.

6.4 Phương pháp và môi trường kiểm thử

Kiểm thử được chia theo tầng để tránh suy rộng kết quả. JVM kiểm tra domain, mapper, policy và ViewModel trong các test đã viết. Build kiểm tra biên dịch/đóng APK. Lint phân tích tĩnh. Firestore/API Emulator thực thi Rules và transaction trên dữ liệu giả; instrumentation kiểm tra layout, SQLite và Activity thật. Tích hợp provider ngoài được ghi thành phạm vi riêng chưa xác minh.

Mọi log mới nằm trong docs/report/evidence, gắn HEAD nền và audit ngày 24/10/2026. Các log lỗi khởi tạo ban đầu vẫn được giữ: Gradle từng bị sandbox chặn cache; Firestore từng lỗi Java loopback; lượt offline từng mất kết nối adb và báo 0 test. Những lượt đó không được tính là qua. Sau điều chỉnh môi trường, lệnh được chạy lại và kết quả mới là cơ sở kết luận.

Bảng 21 tổng hợp kết quả kiểm thử mới trên revision được audit.

Bảng 21 Kết quả kiểm thử mới trên revision được audit

Kiểm tra	Kết quả hiện tại	Giới hạn
JVM Android	253 test; 0 failure; 0 error; 0 skipped	Không phải số chức năng hoặc coverage
assembleDebug + lintDebug	Build qua; 0 lỗi; 693 cảnh báo Lint	Cảnh báo còn cần phân loại
Backend check và unit	Exit 0; 183 qua, 0 lỗi, 0 skipped	Logic local, không provider live
Cloudinary Worker	9 qua, 0 lỗi, 0 skipped	Không upload thật
Firestore/API Emulator	173 qua, 0 lỗi, 0 skipped	Project demo-rescuefarm; bao gồm Hũ
Navigation	59 destination Fragment có class	Kiểm tra tĩnh
Instrumentation chọn lọc	11 qua, 0 lỗi, 0 skipped	9 layout + 1 Room + 1 rotation
Ảnh UI và Room restart	Test chụp actor và 2 method offline qua	Ảnh thật, phạm vi theo manifest

Không cộng các con số ở bảng thành số chức năng hoặc phần trăm coverage. Chẳng hạn 173 test Firestore/API bao gồm nhiều trường hợp cùng một module, còn một bài screenshot mở nhiều màn trong cùng test. ReportScreenshotEvidenceTest được chọn bằng class hoặc method rõ ràng; không chạy toàn bộ instrumentation vào project thật.

6.5 Kết quả Hũ tại các biên quan trọng

Bảng 22 tổng hợp kết quả hũ tại các biên quan trọng.

Bảng 22 Kết quả Hũ tại các biên quan trọng

Kịch bản	Điều kiện hoặc thao tác	Kết quả được kiểm tra
Worker chạy buổi tối	Tạo chuyến khi config cũ expiryHours 12/24	Chuyến mới vẫn allocatedAt + 24 giờ
Biên dưới	Hạn đúng 24 giờ + một phút	Không được chọn vì điều kiện lớn hơn nghiêm ngặt
Biên trên	Hạn đúng 48 giờ hoặc vượt	48 giờ có thể chọn; vượt bị loại
Campaign ngắn	endDate không đủ trọn cửa sổ giao	Không giữ tiền/kho
Gửi ảnh sát hạn	DELIVER trước deadline	Admin VERIFY/PAY sau hạn vẫn hợp lệ
Quá hạn chưa ảnh	RESERVED/ACCEPTED tới hạn	Hủy và giải phóng một lần
Retry/concurrency	Replay callback hoặc phân bổ cạnh tranh	Không cộng tiền/giữ kho hai lần
Chuyến cũ	Policy mới được gọi sau tồn tại chuyến	Không đổi deadline cũ
Múi giờ thiết bị	UTC, Los Angeles, Novosibirsk, Việt Nam	RescueFundTime vẫn hiển thị giờ VN

Các tên test và log của rescue-fund.test.cjs là nguồn kiểm tra chi tiết. Phần này không cộng máy móc "43 test Hũ" của log lịch sử vào 173 test mới: suite mới được đếm từ output của lần chạy hiện tại. Hai test RescueFundTime là một phần trong tổng JVM và được phân tích riêng cho hành vi định dạng.

6.6 Kết quả responsive và offline

ResponsiveLayoutsTest qua chín bài được chọn; OfflineRoomEvidenceTest qua một bài; CourseRotationEvidenceTest qua một bài trên app demo chưa đăng nhập với courseAuditIsolated=true. Cờ này chỉ cho phép test chạy, không tự cấu hình emulator. Trước khi bật cờ, APK demo đã có Firebase project và HTTP localhost riêng.

Bảng 23 tổng hợp kết quả responsive và offline.

Bảng 23 Kết quả responsive và offline

Mức bằng chứng	Đã kiểm tra	Phần chưa suy rộng
Layout	Inflate/measure cấu hình nhỏ, ngang, rộng và font lớn	Không chứng nhận từng Fragment tương tác
Activity	Form login giữ email qua xoay và recreate	Không khẳng định mọi form đều giữ đầy đủ

Mức bằng chứng	Đã kiểm tra	Phần chưa suy rộng
SQLite	Ghi/đóng/mở DB test, đọc domain qua offline repository	Không thay UI test và ảnh
UI Room	Tiến trình mới, SDK persistence off, network disabled	Catalog chữ; chưa chứng minh ảnh disk cache
Khôi phục mạng	Xóa cờ offline, enableNetwork	Chưa đo toàn bộ sync khi mạng chập chờn

Điểm mạnh của thử nghiệm UI Room là loại ảnh hưởng cache Firestore cũ ở tiến trình trước. Tuy vậy, chỉ catalog demo được assert; cache đơn riêng và chuyển tài khoản cần thêm kịch bản UI độc lập. TTL trong code không đồng nghĩa toàn bộ cache tự bị xóa đúng giây hết hạn. Các giới hạn này quan trọng hơn một kết luận chung "app hỗ trợ offline hoàn toàn".

6.7 Rủi ro còn lại và kết luận kỹ thuật

Hệ thống có kiến trúc Android Java/XML kết hợp Firestore, Room và backend tin cậy phù hợp bài toán thương mại điện tử giải cứu. Sự tách Product–Batch, kiểm tra tồn trong transaction, request identity và ledger Hũ là các quyết định thiết kế giúp giải thích những trường hợp cạnh tranh và retry. Bộ test mới cung cấp bằng chứng có giới hạn cho những hành vi được nêu trong ma trận.

Những phần chưa xác minh gồm Google OAuth thật, upload/camera và chứng từ provider thật, FCM tới thiết bị ngoài môi trường test, callback SePay live và trạng thái deploy production. Không có phép đo tải, thời gian phản hồi hoặc tác động môi trường để công bố số liệu. Lint còn cảnh báo cần được ưu tiên theo mức ảnh hưởng, không thể gọi mã hoàn hảo chỉ vì build thành công.

Về học thuật, bản báo cáo theo cấu trúc mẫu PDF người dùng cung cấp và giữ Chương 1, nhưng MSSV, lớp cụ thể, phân công được giao và tỷ lệ hoàn thành cần nhóm xác nhận. Chưa chạy hệ thống kiểm tra tương đồng UEH ở chế độ mặc định; không tuyên bố dưới 20%. Trước nộp, nhóm phải đối chiếu thông tin, kiểm tra PDF và lưu kết quả tương đồng thật.

PHỤ LỤC PHÂN CÔNG VÀ MINH CHỨNG GIT

Nguồn mã và lịch sử tích hợp

Mã đầy đủ được chia sẻ tại https://github.com/LaiThanhDat-glitch/NHOM_7_MobileApp và bản remote cộng tác https://github.com/datweb07/NHOM_7_MobileApp. HEAD nền của audit là 28adb48bb2e5e4ac8230379a1152f64a52251f5e. git-evidence.json lưu merge SHA, ngày, tên branch/PR trong commit và đường dẫn module; không coi số commit hay LOC là tỷ lệ đóng góp.

Bảng 24 tổng hợp các merge gần head để kiểm tra tích hợp.

Bảng 24 Các merge gần HEAD để kiểm tra tích hợp

Merge SHA	Branch hoặc PR trong lịch sử	Sản phẩm tra cứu
28adb48	Merge pull request #197 from datweb07/main	app/src/main/java/com/rescuefarm/ui/fund/RescueFundFragment.java, app/src/main/java/com/rescuefarm/ui/fund/RescueFundTime.java, app/src/test/java/com/rescuefarm/ui/fund/RescueFundTimeTest.java
3e59e15	Merge pull request #196 from datweb07/main	app/src/androidTest/java/com/rescuefarm/CourseRotationEvidenceTest.java, app/src/androidTest/java/com/rescuefarm/OfflineRoomEvidenceTest.java, app/src/main/res/layout-w840dp/fragment_product_detail.xml
bf648f2	Merge pull request #195 from datweb07/main	app/src/androidTest/java/com/rescuefarm/RequestedUiTest.java, app/src/main/java/com/rescuefarm/ui/common/HomeSearchInput.java, app/src/main/java/com/rescuefarm/ui/home/GuestHomeFragment.java
b6ee6b8	Merge pull request #194 from datweb07/main	app/src/main/java/com/rescuefarm/ui/admin/AdminDashboardFragment.java, app/src/main/java/com/rescuefarm/ui/admin/AdminResourceFragment.java, app/src/main/java/com/rescuefarm/ui/fund/RescueFundFragment.java
5553b66	Merge pull request #193 from datweb07/main	app/src/main/java/com/rescuefarm/ui/fund/RescueFundFragment.java, app/src/main/java/com/rescuefarm/ui/fund/RescueFundUi.java, app/src/main/res/values/rescue_fund.xml

Merge SHA	Branch hoặc PR trong lịch sử	Sản phẩm tra cứu
3e80abd	Merge pull request #192 from nguyentankhiem1610/main	app/src/androidTest/java/com/rescuefarm/RescueFundPresentationTest.java, app/src/main/java/com/rescuefarm/ui/fund/RescueFundFragment.java, app/src/main/java/com/rescuefarm/ui/fund/RescueFundUi.java

Nhiều alias xuất hiện trong shortlog. Cần nhóm xác nhận việc gộp datweb07 với Đạt Trương, LaiThanhDat-glitch với Lại Thành Đạt, chinhngprit với Nguyễn Phương Chinh và các alias khác; bảng phân công không tự quy đổi chúng thành phần trăm. Tên thành viên lấy từ bìa mẫu và README, không tự bịa MSSV.

Bảng phân công chờ xác nhận của nhóm

Bảng 25 tổng hợp phân công cần nhóm xác nhận trước nộp.

Bảng 25 Phân công cần nhóm xác nhận trước nộp

Thành viên	Nhiệm vụ và bàn giao	Module và merge	Trạng thái và phần trăm
Lại Thành Đạt	Chờ nhóm xác nhận nhiệm vụ được giao và sản phẩm đã duyệt	Tra git-evidence; alias và sở hữu module chờ xác nhận	Chờ xác nhận
Phan Khắc Anh Tuấn	Chờ nhóm xác nhận nhiệm vụ được giao và sản phẩm đã duyệt	Tra git-evidence; alias và sở hữu module chờ xác nhận	Chờ xác nhận
Nguyễn Phương Chinh	Chờ nhóm xác nhận nhiệm vụ được giao và sản phẩm đã duyệt	Tra git-evidence; alias và sở hữu module chờ xác nhận	Chờ xác nhận
Nguyễn Tấn Khiêm	Chờ nhóm xác nhận nhiệm vụ được giao và sản phẩm đã duyệt	Tra git-evidence; alias và sở hữu module chờ xác nhận	Chờ xác nhận
Trương Thành Đạt	Chờ nhóm xác nhận nhiệm vụ được giao và sản phẩm đã duyệt	Tra git-evidence; alias và sở hữu module chờ xác nhận	Chờ xác nhận

Nhiệm vụ được giao, sản phẩm bàn giao được duyệt và phần trăm hoàn thành là thông tin nhóm cần xác nhận. Module/merge được liệt kê trong evidence giúp tra cứu, không thay thế một biên bản phân công. Kịch bản demo chi tiết và khoảng 15 câu hỏi bảo vệ được bàn giao ở demo-5-minutes.md và defense-questions.md để nhóm phân vai và diễn tập.

TÀI LIỆU THAM KHẢO

[1] Android Developers. Save data in a local database using Room. https://developer.android.com/training/data-storage/room. Truy cập 24/10/2026. Cơ sở phân biệt Room với SQLite và cache đọc trên thiết bị.

[2] Firebase. Transactions and batched writes. https://firebase.google.com/docs/firestore/manage-data/transactions. Truy cập 24/10/2026. Cơ sở giải thích transaction; bằng chứng logic RescueFarm nằm trong code và test của repo.

[3] Firebase. Connect your app and start prototyping with Local Emulator Suite. https://firebase.google.com/docs/emulator-suite/connect_and_prototype. Truy cập 24/10/2026. Cơ sở dùng project demo và kết nối emulator theo từng SDK.

[4] Android Developers. App resources overview. https://developer.android.com/guide/topics/resources/providing-resources. Truy cập 24/10/2026. Cơ sở giải thích tài nguyên thay thế theo cấu hình màn hình.

[5] Firebase. Access data offline. https://firebase.google.com/docs/firestore/manage-data/enable-offline. Truy cập 24/10/2026. Cơ sở phân biệt persistence SDK với Room riêng của app.

[6] Firebase. Authenticate with Google on Android. https://firebase.google.com/docs/auth/android/google-signin. Truy cập 24/10/2026. Cơ sở cho luồng Google credential tới Firebase Auth.

[7] Android Developers. Task scheduling. https://developer.android.com/develop/background-work/background-tasks/persistent. Truy cập 24/10/2026. Cơ sở mô tả WorkManager và giới hạn thời gian thực thi.

[8] SePay Developer. Xác thực webhook SePay thế nào. https://developer.sepay.vn/vi/sepay-webhooks/xac-thuc. Truy cập 24/10/2026. Cơ sở xác thực webhook; không phải chứng cứ RescueFarm đã nhận tiền thật.

[9] Nhóm RescueFarm. Repository và mã tại commit 28adb48bb2e5e4ac8230379a1152f64a52251f5e cùng diff thực nghiệm trong bàn giao, 24/10/2026. Nguồn triển khai chính: app/src/main, functions, cloudinary-worker, firestore.rules và test. Đường dẫn Java rút gọn theo quy ước đầu báo cáo, backend tính từ root repo để tra cứu trên GitHub.

[10] Nhóm RescueFarm. MOBILE FINAL PROJECT.pdf, tài liệu người dùng cung cấp, 13 trang. Nguồn bìa, thông tin học phần, Chương 1 và cấu trúc sáu chương. Không tự suy rằng bản mẫu là quy định định dạng đầy đủ của trường khi tài liệu chưa nêu các thông số đó.